大模型算法岗 · 三项目实战

电商客服 SFT+DPO | 金融研报 RAG | vLLM 推理部署

项目文档：项目介绍 · 简历描述 · 面试方法 · 常见 QA · 启动方法 · 实验数据

源码目录：project1_ecom_cs_sft_dpo / project2_fin_report_rag / project3_llm_serving
说明：文中所有实验数字均由仓库内脚本实际运行得到，并标注运行环境；8B 训练与 vLLM 部署的指标来自 5090 单卡实测，项目一数据管线、规则基线与 0.6B 复现、项目二全部检索实验、项目三网关逻辑验证来自本机（Apple M5 Pro）实测。

目 录

第〇部分 总览与阅读说明 5
0.1 先读这个：数字从哪来、哪些能写进简历 5
0.2 三个项目是一条线 5
0.3 执行节奏（12 周 = 3 个月） 5
0.4 仓库结构与交付物 6
项目一 电商客服问答 —— Qwen3-8B 的 SFT + LoRA 消融 + DPO 偏好对齐 7
一、项目介绍 7
1.1 项目一句话简介 7
背景与目标 7
技术方案总览 7
1.2 简历直接可用的项目描述（三种不同表述，已降重） 7
1.3 面试直接可背的项目介绍（1-2 分钟） 9
1.4 项目英文简历描述 9
二、面试方法 10
2.1 引导提问的思路、方向与话术 10
2.2 追问应对、诚实边界与如何化解深挖 10
2.3 面试常见 QA（45 题） 11
三、项目启动方法 14
3.1 环境 14
3.2 一步步启动 15
3.3 常见报错 15
四、项目实验数据 15
4.1 数据构建与清洗（本机实测） 15
4.2 基线与评测脚本自检（本机实测） 16
4.3 Qwen3-0.6B 小规模复现（本机实测，Apple M5 Pro / MPS） 16
4.4 Qwen3-8B 正式实验（5090 单卡实测） 18
4.5 面试时怎么讲实验结论 19
项目二 金融研报问答 —— 多栏 PDF 解析 + 混合检索 + 重排 + 证据约束生成 20
一、项目介绍 20
1.1 项目一句话简介 20
背景：为什么做 RAG，而且选金融 20
数据说明 20
技术选型 20
1.2 简历直接可用的项目描述（三种不同表述，已降重） 21
1.3 面试直接可背的项目介绍（1-2 分钟） 22

1.4 项目英文简历描述 . . . 22
二、面试方法 . . . 23
2.1 引导提问的思路、方向与话术 . . . 23
2.2 追问应对、诚实边界与如何化解深挖 . . . 23
2.3 面试常见 QA（45 题） . . . 23
三、项目启动方法 . . . 27
四、项目实验数据 . . . 27
4.1 解析质量 . . . 27
4.2 切块网格（Recall@5） . . . 27
4.3 多路召回与 Query 改写（chunk 512 / overlap 50） . . . 28
4.4 FAISS 索引对比 . . . 29
4.5 重排与拒答 . . . 29
4.6 端到端（0.6B 为 CPU 链路验证；最后一行 Qwen3-8B 走项目三 vLLM 服务） . . . 29
4.7 故障记录（面试必讲） . . . 30
项目三 推理部署 —— vLLM + 量化 + FastAPI 网关 + 压测与故障演练 . . . 31
一、项目介绍 . . . 31
1.1 项目一句话简介 . . . 31
背景：为什么一定要有部署项目 . . . 31
推理框架与技术选型 . . . 31
1.2 简历直接可用的项目描述（三种不同表述，已降重） . . . 31
1.3 面试直接可背的项目介绍（1-2 分钟） . . . 33
1.4 项目英文简历描述 . . . 33
二、面试方法 . . . 34
2.1 引导提问的思路、方向与话术 . . . 34
2.2 追问应对、诚实边界与如何化解深挖 . . . 34
2.3 面试常见 QA（45 题） . . . 34
三、项目启动方法 . . . 37
四、项目实验数据 . . . 38
4.1 KV Cache 容量估算（确定性计算，可直接引用） . . . 38
4.2 量化对比（5090 单卡实测） . . . 38
4.3 并发压测（真实 vLLM，5090 单卡实测） . . . 38
4.4 网关逻辑验证（mock 后端，本机实测；仅验证逻辑，不代表 GPU 性能） . . . 39
4.5 故障记录表 . . . 39
4.6 性能优化顺序（面试可直接说） . . . 40
附录 通用经验：踩坑、简历自检、开源仓库 . . . 41
A.1 我踩过的坑，你们别再踩了 . . . 41
A.2 做完之后怎么判断能不能写进简历（5 条自检） . . . 41

A.3 讲 badcase 的万能结构 41
A.4 开源仓库汇总 41
A.5 三个项目合在一起时的简历排版建议 42

第〇部分 总览与阅读说明

0.1 先读这个：数字从哪来、哪些能写进简历

> **诚实边界（非常重要）**
> 本文档的每一个数字都来自仓库内脚本的真实运行结果（文档构建脚本 docs/build_doc.py 直接读取各项目 results/ 下的 JSON 填表）。
> 分三类，写简历和面试时请严格区分：
> ① 【本机实测】：Apple M5 Pro（无 NVIDIA GPU）上实际跑出的数字。项目一的数据清洗统计与规则基线、项目一 Qwen3-0.6B 小规模复现、项目二全部检索/重排/拒答实验、项目三 KV Cache 计算与网关逻辑验证都属于这一类。
> ② 【5090 实测】：在 5090 单卡上跑 Qwen3-8B / vLLM 得到的数字（8B 的 SFT/DPO 效果、量化对比、真实 QPS/P95），
> 见项目一 4.4、项目二 4.6 最后一行、项目三 4.2–4.3。这批数字是简历里最硬的部分，要能说出训练耗时、显存峰值、badcase 条数这些细节。
> ③ 【mock 模拟】：项目三网关在 mock 后端上的压测/故障演练数字，只能证明"熔断、降级、限流、缓存的逻辑按设计工作"，不代表 GPU 性能，不要写进简历的性能指标里。

这样做的原因很简单：面试官追问三层就会露馅的数字，写上去反而减分（见附录"做完之后怎么判断能不能写进简历"）。本仓库的设计目标是：你在 5090 上按脚本跑一遍，得到的每个数字都能讲清楚"怎么来的、为什么是这个值、换个设置会怎样"。

0.2 三个项目是一条线

三个项目不是三份独立作业，而是一条完整链路：电商客服里你微调了模型（训练与对齐）→ 金融研报里你做了知识检索（RAG）→ 部署上线时把两者串到一个在线服务里（推理优化与工程化）。面试官最想看到的就是：你能不能从一个具体需求出发，把训练、检索、上线这条路走通。

```
用户请求 -> FastAPI 网关 — 限流(令牌桶) — 缓存(Redis, 热点预热) — 命中直接返回
                  |
                  ├— 意图路由: 客服类 —> vLLM[项目一 SFT+DPO 模型, AWQ-INT4]
                  └— 意图路由: 研报类 —> RAG 服务[项目二: 改写→FAISS+BM25→bge-reranker→约束校验]
                                  └—> vLLM 生成(证据绑定 + 引用校验)
                超时/熔断 —> 降级小模型 —> 兜底话术 + 转人工
                日志(input/output tokens, latency) + /stats + Prometheus
```

项目	方向	核心技术	必须有的实验	最能拉开差距的点
一 电商客服	训练与对齐	Qwen3-8B、TRL、PEFT LoRA/QLoRA、DPO	rank 消融、target_modules、QLoRA vs LoRA、SFT vs DPO、β、偏好噪声	数据清洗统计 + 翻车案例（复读机/变卦/事实错误）
二 研报 RAG	检索增强	pdfplumber 版面解析、BGE、FAISS、BM25、bge-reranker	切块网格、多路召回、索引对比、重排、拒答阈值	多栏/表格解析 + 证据级评测集 + 拒答设计
三 推理部署	推理优化与工程	vLLM、AWQ/GPTQ、FastAPI、Redis、Prometheus	量化对比、并发阶梯压测、参数对比、故障演练	能报出 QPS/P95/显存 + 讲清瓶颈与止血 SOP

0.3 执行节奏（12 周 = 3 个月）

周	项目一：训练与对齐	项目二：RAG 系统	项目三：推理部署
W1	客服数据清洗 + SFT 跑通	—	—

周	项目一：训练与对齐	项目二：RAG 系统	项目三：推理部署
W2	LoRA 消融实验	—	—
W3	客服偏好数据 + DPO	—	—
W4	客服 badcase + 文档	—	—
W5	—	研报 PDF 解析 + 切块 + baseline	—
W6	—	金融 QA 评测集 + 多路召回实验	—
W7	—	重排 + 故障处理	—
W8	—	Demo + 评测文档	—
W9	—	—	客服模型量化实验
W10	—	—	服务化 + 压测
W11	—	—	工程化 + 故障演练
W12	—	—	客服 + 研报三项目串联 + 总文档

前 1 个月补基础（Transformer、训练闭环，建议先花 2 天读 nanoGPT 源码），中间 3 个月做这三个项目，后 2 个月刷题 + 面试。算力：默认单卡 5090（32GB）。三个项目都按“不吃算力”设计：8B 用 QLoRA 单卡可训，推理用 AWQ-INT4 单卡可服务。

0.4 仓库结构与交付物

```
LLM_Projects/
├── project1_ecom_cs_sft_dpo/   # 项目一：数据构造/清洗 → SFT(LoRA/QLoRA) → 偏好数据 → DPO → 评测 → badcase → 合并
├── project2_fin_report_rag/    # 项目二：演示语料 → 版面解析 → 切块 → 混合检索 → 重排 → 约束生成 → 评测 → Demo/API
├── project3_llm_serving/       # 项目三：量化 → vLLM → 网关(缓存/限流/路由/熔断降级/监控) → 压测 → 故障演练
├── docs/                       # 本文档的生成脚本（python docs/build_doc.py 可随实验结果重新生成）
└── 项目文档.pdf
```

项目一 电商客服问答 —— Qwen3-8B 的 SFT + LoRA 消融 + DPO 偏好对齐

一、项目介绍

1.1 项目一句话简介

> 针对电商平台日均 5000+ 客服咨询（退换货/物流/商品参数/售后投诉）中规则引擎只能自动解决约三成的问题，基于 Qwen3-8B 构建“数据清洗 → LoRA/QLoRA 微调 → 偏好数据构建 → DPO 对齐 → 分桶评测与 badcase 迭代”的完整后训练链路，让模型接住高频问题、少转人工、不复读、不说错政策。

背景与目标

- 业务现状：线上关键词规则引擎只覆盖历史高频问法；用户换个说法、问题依赖订单上下文（签收天数、类目、物流状态）时就答错或直接转人工。本仓库复现的规则基线在留出问法测试集上自动解决率只有 28.0%（准确率 33.5%），在训练分布内问法上为 48.2%——“规则写死、泛化不到新问法”是可以量化的。
- 目标：自动解决率 30%~40% → 70%+；政策错误率接近 0；该转人工（赔偿认定、威胁投诉）的必须转。
- 为什么选客服：数据好拿（公开对话 + FAQ + SOP 模板扩写）；指标硬（自动解决率、准确率、满意度、转人工率，有说服力）；SFT 教“怎么回答”、DPO 教“哪种回答更好”（“好答案”=给政策依据+操作路径，“差答案”=只说请联系人工），场景天然适配；字节、阿里、京东、拼多多都有同类业务，面试官容易代入。

技术方案总览

环节	做法	代码
业务知识	SOP 唯一事实源：20 个意图 × 6 大业务域，政策数字、商品库、问法模板（训练/测试问法严格隔离）	src/ecs/sop.py
数据构造	SOP 模板扩写 + 公开数据转换 + 可选 GPT 改写；多轮（突然变卦/追问）占 10%；刻意保留真实噪声	scripts/00、01
数据清洗	PII 脱敏 → 短回复过滤(<20字) → 政策冲突检测(旧版 SOP 数字) → 精确去重 → MinHash 近重复；12000 → 5619 条	src/ecs/clean.py
SFT	TRL SFTTrainer + PEFT，completion-only loss，Qwen3 chat template 关闭 thinking，QLoRA NF4 单卡 32GB	scripts/03
偏好数据	SFT 模型对每个 query 采样 3 个回答 → 规则打分 → 分差不足进人工标注队列；冷启动用规则扰动生成 5 类差回复	scripts/04、05
DPO	TRL DPOTrainer；同一 SFT adapter 加载两份（policy 可训 / reference 冻结），无需合并、单卡可跑	scripts/06
评测	留出问法测试集 400 条 + 4 个困难子集；准确率/自动解决率/满意度/复读率/转人工率；可选 LLM-as-judge	scripts/07、src/ecs/metrics.py
分析	8 类 badcase 自动分桶 + 两模型修复/退化/胜平负对比；合并 LoRA 给项目三部署	scripts/08、09

1.2 简历直接可用的项目描述（三种不同表述，已降重）

> **使用说明**
>
> 下面三版里的 8B 指标都取自第四章的实测表（5090 单卡 QLoRA，5339 条 × 3 epoch；评测为 400 条留出问法测试集、贪心解码），可以直接往简历上抄；0.6B 小规模复现的数字也已标注为本机实测。三版用词、句式、侧重点均不同：版本 A 偏数据与实验、版本 B 偏对齐与评测、版本 C 偏业务落地，可按投递岗位选择。

版本 A | 电商客服大模型后训练（SFT + LoRA 消融 + DPO 对齐）

项目定位：

面向电商售后高频咨询（退换货/物流/商品参数/投诉），线上规则引擎仅能自动解决约三成新问法，目标是让 8B 模型稳定接住高频问题、减少转人工

核心方案：

- 构建 SFT 数据管线：SOP 模板扩写 + 公开对话转换，经 PII 脱敏、短回复过滤、旧版政策冲突检测、精确去重 + MinHash 近重复，原始 12000 条清洗为 5619 条
- 基于 TRL + PEFT 在 Qwen3-8B 上做 QLoRA 微调，完成 rank(4/16/64)、target_modules(q,v / all-linear)、QLoRA vs LoRA 三组消融
- SFT 模型多采样 + 规则打分 + 人工复核构建 2000 对客服偏好数据，进行 SFT-only vs SFT+DPO 对照及 β、偏好噪声(10%/20%/30%)实验
- 建立留出问法测试集与 4 个困难子集（超政策窗口/多轮变卦/追问/需转人工），8 类 badcase 自动分桶驱动数据迭代

实验与结果：

- 自动解决率由规则引擎 28.0% 提升至 73.5%（SFT）/79.0%（SFT+DPO），准确率提升至 81.5%，复读率降至 0.2%
- rank/模块消融（0.6B 复现实测）：r=4/16/64 自动解决率 26.8%/52.0%/66.8%，all-linear(r=16) 达 75.0%；8B 上 r=16 与 r=64 差距 1.0 个点
- 在 Qwen3-0.6B 上完成全流程小规模复现：自动解决率 28.0%（规则）→ 52.0%（SFT）；发现合成偏好对导致 DPO 过优化（降至 22.5%），通过 on-policy 采样 + 降学习率 + RPO 缓解至 51.5%

技术栈：

PyTorch、Transformers、TRL、PEFT(LoRA/QLoRA)、DeepSpeed ZeRO-2、vLLM、Qwen3-8B

版本 B | 智能客服模型偏好对齐与评测体系

项目定位：

围绕“政策准确、给出操作路径、该转人工才转人工”三个目标，为电商客服场景搭建可复现的 SFT→DPO 后训练与评测闭环

核心方案：

- 以业务 SOP 为唯一事实源统一生成训练数据、评测要点与禁止表述，训练/测试问法模板物理隔离，避免模板泄漏导致评测虚高
- 定义 5 类典型差回复（只推人工/含糊/旧政策/复读/答非所问）作为 rejected，结合 SFT 采样候选的自动打分与人工标注队列构造偏好对
- 采用“同一 adapter 双份加载”的 DPO 实现（policy 可训、reference 冻结），QLoRA 下单卡 32GB 完成训练；对比 β=0.05/0.1/0.5
- 设计否定感知的禁止表述匹配、6-gram 复读检测、满意度代理分与可选 LLM-as-judge，按业务域与困难子集分桶报告

实验与结果：

- SFT+DPO 相比 SFT-only：转人工率下降 1.4 个点、满意度 3.75→3.88（+0.13/5），盲测胜率 62%
- 0.6B 复现中定位 DPO 过优化：合成偏好对使自动解决率 52.0%→22.5%，改为 on-policy 偏好对 + 降学习率 + RPO 后恢复至 51.5%（本机实测）
- 规则基线自动解决率 28.0%（本机实测），作为对照锚点

技术栈：

TRL(SFTTrainer/DPOTrainer)、PEFT、bitsandbytes、vLLM、Pandas、W&B

版本 C | 电商售后智能问答模型落地（训练侧）

项目定位：

将电商售后客服从“关键词规则 + 人工兜底”升级为“微调大模型 + 规则兜底”，在单卡 32GB 资源约束下完成训练、对齐与上线前评估

核心方案：

- 梳理退换货、物流、商品参数、投诉、售后、优惠 6 大域 20 类意图，把订单上下文（类目/签收天数/物流状态/运费险）注入输入，解决“同一问题不同订单答案不同”
- 数据侧重点治理旧版 SOP 混入与“好的亲”式无效回复，规则化统计每步过滤量（12000→5619），过滤样本留档抽检
- 训练侧以 QLoRA r=16 为主配置，辅以 rank/模块/精度消融确定性价比最优点；DPO 专门压制“能答却推人工”和“该转却不转”
- 产出合并后的模型供推理服务 AWQ 量化部署，并用同一测试集验证量化掉点

实验与结果：

- 自动解决率 79.0%、准确率 81.5%（规则基线 28.0%），满意度 3.88/5；需转人工子集召回 88.0%
- 形成数据清洗报告、消融实验表、badcase 分桶报告，可一键复现

技术栈：

Python、PyTorch、Transformers、TRL、PEFT、LLaMA-Factory(对照)、DeepSpeed

1.3 面试直接可背的项目介绍（1-2 分钟）

约 1 分 40 秒

我做的第一个项目是电商客服的大模型后训练。背景是电商客服每天 5000 多条咨询，主要是退换货、物流、商品参数和投诉，原来的规则引擎只能覆盖历史高频问法，我复现的规则基线在新问法测试集上自动解决率只有 28.0%，剩下全转人工。

我的目标是用 Qwen3-8B 把这部分接住。做法分四步：第一步是数据，我以业务 SOP 为唯一事实源，用模板扩写加公开对话拼出原始 12000 条数据，然后做了脱敏、短回复过滤、旧版政策冲突检测、精确去重和 MinHash 近重复，最后剩 5619 条——这里我特别处理了“旧版政策混进训练集”的问题，因为它会让模型说出过期的退货天数。

第二步是 SFT，用 TRL 加 PEFT 做 QLoRA，单卡 32G 能跑，我做了 rank、target_modules 和 QLoRA 对 LoRA 三组消融。有个和教程说法不一样的发现：在 0.6B 小模型、训练步数有限的设置下，rank 越大越好，r=4 到 64 自动解决率从 26.8% 涨到 66.8%，把 LoRA 加到所有线性层效果最好，说明小模型上容量是瓶颈；8B 上 rank 的边际收益明显变小，r=16 到 r=64 自动解决率只差 1.0 个点，所以 8B 就选 r=16。

第三步是 DPO，SFT 之后模型最大的问题是“能答却说请联系人工”和复读，所以我让 SFT 模型对每个问题采样 3 个回答，按规则打分、分差不够的进人工标注，构造偏好数据做 DPO。这里我踩过一个坑：一开始用规则造的“差回答”做 rejected，训练时奖励准确率接近 100%，但测试集反而大幅下降，模型回答流畅却开始编内容——偏好对太容易导致过优化。后来改成让 SFT 模型自己采样候选再打分成对、降学习率、并在 DPO loss 上叠加 SFT loss，才把效果拉回来。

第四步是评测和 badcase，测试集的问法和训练集完全隔离，还专门做了超政策时效、多轮变卦、需要转人工这几个困难子集，按 8 类错误自动分桶。最后把模型合并给推理服务做量化部署。整个项目我最大的收获是：数据质量和评测口径比调参重要得多。

1.4 项目英文简历描述

English version

E-commerce Customer-Service LLM Post-Training (SFT + LoRA Ablation + DPO)

• Built an SFT data pipeline for after-sales QA (returns, logistics, product specs, complaints): SOP-template expansion plus public dialogues, PII masking, short-reply filtering, outdated-policy conflict detection, exact and MinHash near-duplicate removal (12000 → 5619 samples).

• Fine-tuned Qwen3-8B with TRL + PEFT (QLoRA, single 32GB GPU); ran ablations on LoRA rank (4/16/64), target modules (q,v vs. all-linear) and QLoRA vs. LoRA.

• Constructed 2,000 preference pairs from multi-sample SFT outputs with rule-based scoring and human review; compared SFT-only vs. SFT+DPO, β (0.05/0.1/0.5) and preference-label noise (10–30%).

• Designed a held-out evaluation set with hard subsets (out-of-policy-window, multi-turn intent change, escalation) and an 8-type bad-case taxonomy; raised auto-resolution from 28.0% (rule engine) to 79.0% and cut repetition to 0.2%.

Tech: PyTorch, Transformers, TRL, PEFT, bitsandbytes, DeepSpeed, vLLM

二、面试方法

2.1 引导提问的思路、方向与话术

面试官的问题有 70% 是从你自我介绍里“埋的钩子”展开的。原则：把你准备最充分的点主动说出一半，让面试官来问另一半。

想引导到的方向	在介绍里埋的钩子（原话）	对方大概率追问	你准备好的“另一半”
数据质量	“我特别处理了旧版政策混进训练集的问题”	怎么发现的？怎么检测？	badcase 里模型说“15 天无理由”→ 回溯训练数据 → 写政策数字正则对比当前 SOP；被过滤样本留档抽检
消融实验	“小模型上 rank 越大越好，和‘r=8 就够’的说法不一样”	为什么？8B 上呢？	0.6B：r=4/16/64 = 26.8%/52.0%/66.8%，all-linear 75.0%；小模型底座能力弱，需要更多可训练容量；8B 底座本身会说话，低秩即可——8B 上 r=16 与 r=64 只差 1.0 个点，但参数量是 4 倍、耗时多 10min
DPO 动机	“SFT 之后最大问题是能答却推人工”	为什么 DPO 不用 PPO？	无需 RM、更稳、数据量要求低；32GB 跑 PPO 四模型不现实
评测口径	“测试集问法和训练集完全隔离”	为什么要隔离？	规则基线训练分布内 48.2% vs 新问法 28.0%，不隔离就看不出泛化
翻车经历	“DPO 训练指标很好看，测试集却掉了一大截”	为什么？怎么修的？	合成偏好对太容易→过优化；on-policy 偏好对 + 降 lr + RPO；β 和噪声实验作为旁证
评测排障	“有一次所有模型准确率都是 0”	怎么排查的？	看原始输出全是“!!!!”→ MPS 上 SDPA 长左填充数值异常 → eager + fp32

常用引导话术

- “这里面其实有个挺有意思的发现，……（停顿）”—— 让面试官说“展开讲讲”。
- “这个问题我当时也纠结过，最后用实验决定的。”—— 把主观题拉回你的实验数据。
- “我先说结论，再说我是怎么验证的。”—— 结论先行，控制节奏。
- 被问到没做过的方向（如 PPO/GRPO）：“这个我没有实际跑，但我在选型时对比过，……，如果要做我会先……”—— 把话题拉回你的选型依据。

2.2 追问应对、诚实边界与如何化解深挖

诚实边界：哪些是你做的，哪些不是

事实	可以怎么说	千万别说
数据由 SOP 模板扩写 + 公开数据 + GPT 改写，非真实线上日志	“训练数据一部分来自公开电商对话和 FAQ，一部分是基于业务 SOP 模板扩写并用 GPT 改写问法，原始约 1.2 万条清洗到约 5.6 千条”	“用的公司真实客服日志”
满意度是规则代理分 / LLM-as-judge，不是真实用户打分	“满意度用规则代理分，并抽样做了 LLM 打分和人工复核，二者趋势一致”	“用户满意度提升了 X”
没有真实线上 A/B	“离线评测 + 小规模盲测（人工/LLM 两两比较）”	“上线后转人工率降了 X”
PPO/GRPO 没跑	“做了适用边界分析，在当前数据规模和单卡资源下选择 DPO”	“PPO 效果不如 DPO”
8B 指标来自 5090 实测（4.4）	“8B 在 5090 上跑过，数字是实测的”，并能说出 loss 曲线、显存峰值、耗时与 badcase 条数	没跑就写数字

化解深挖的三板斧

1. **承认 + 边界 + 方案**：“这一点我没在生产上验证过（承认），我的实验只覆盖离线测试集（边界），如果上线我会先灰度 5% 流量看转人工率和投诉率（方案）。”
2. **把问题拉回数据**：被问“为什么效果提升了”，不要讲玄学，讲“哪个子集提升了多少、修复了哪类 badcase”。
3. **不会的就讲推理过程**：比如被问 DPO loss 推导细节记不清，可以从“Bradley-Terry 偏好模型 + 用策略与参考模型的对数概率比表示隐式奖励”讲起，面试官看的是理解而非背诵。

高频追问三层展开（示例）

第一层	第二层	第三层
LoRA rank 怎么选？	为什么 r=64 没提升？	如果任务换成学新知识（如新商品库）呢？——知识注入需要更大容量/更多模块，或者走 RAG
偏好数据怎么构建？	质量差会怎样？	怎么发现标注噪声？——看 reward margin 分布、chosen/rejected 长度偏差、抽检一致率
为什么选 DPO？	β 调大调小会怎样？	DPO 有什么缺陷？——对 OOD 回答没约束、可能拉低 chosen 概率、长度偏好；可用 IPO/SimPO/加 SFT loss 缓解

2.3 面试常见 QA（45 题）

Q1. 用一句话介绍这个项目。

基于 Qwen3-8B 做电商客服的 SFT+DPO 后训练，把规则引擎只能自动解决三成的高频售后问题提升到七成以上，重点在数据清洗、消融实验和偏好对齐。

Q2. 数据是从哪来的？

三部分：公开电商客服对话/FAQ（转换脚本 00_convert_public.py）、基于业务 SOP 的模板扩写（覆盖 6 域 20 意图并注入订单上下文）、可选 GPT 改写问法提升多样性。原始 12000 条，清洗后 5619 条。

Q3. 清洗具体做了哪些步骤？每步过滤多少？

PII 脱敏 → 短回复(<20 字)过滤 → 旧版政策冲突检测 → 精确去重（归一化 query+上下文）→ MinHash 近重复。本机实测：12000→短回复后 10999→政策冲突后 10474→精确去重后 6222→近重复后 5619。

Q4. 为什么要做“政策冲突检测”？

客服知识有版本，旧 SOP（比如 15 天无理由）混进训练集会让模型说出过期政策，这是最严重的事实错误。我用正则抽取答案中的政策数字，与当前 SOP 对比，不一致直接剔除。

Q5. MinHash 去重原理？阈值怎么定？

把文本切成字符 3-gram shingle，用多个哈希函数取最小值组成签名，签名相同的概率等于 Jaccard 相似度；用 LSH 分桶找候选再精确算 Jaccard。阈值 0.85，抽检发现再低会误删语义不同但字面接近的问题。

Q6. 训练集和测试集怎么划分？如何防止泄漏？

测试集使用独立的问法模板（test_templates），与训练问法物理隔离；同时多轮追问话术也分开。规则基线在训练分布内 48.2%、在留出问法上 28.0%，这个差距就是“泛化”的证据。

Q7. 为什么要把订单上下文放进输入？

同一句“能退吗”，答案取决于类目（食品拆封不退）、签收天数（超 7 天不支持无理由）。不给上下文模型只能瞎猜，规则引擎也正是在这里大面积失败（beyond_window 子集准确率 0）。

Q8. SFT 时 loss 怎么算？为什么只算回答部分？

用 prompt-completion 格式，completion_only_loss=True，只对 assistant 回复计算交叉熵。对 prompt 计算 loss 会让模型学习复述用户问题和 system prompt，浪费容量且可能导致格式混乱。

Q9. Qwen3 的 thinking 模式怎么处理？

客服要求低延迟短回答，训练/推理统一用 apply_chat_template(enable_thinking=False)，并且训练、评测、部署共用一个 prompts.py，保证模板完全一致。

Q10. LoRA 原理？为什么能省显存？

冻结原权重 W，只学低秩增量 ΔW=BA（A:r×d，B:d×r），可训练参数从 d^2 降到 2dr；梯度和优化器状态只存 LoRA 参数，所以显存主要是权重+激活。

Q11. LoRA 的 alpha 有什么用？

缩放系数 alpha/r 控制增量幅度。我固定 alpha=2r，这样改 rank 时有效学习率大致不变，消融才公平。

Q12. rank 怎么选？

用消融决定。0.6B 复现中 r=4/16/64 自动解决率 26.8%/52.0%/66.8%，rank 越大越好——小模型、300 步短训练下可训练容量是瓶颈。8B 底座能力强，增量更“低秩”，r=16 与 r=64 只差 1.0 个点，而 r=64 的参数量是 r=16 的 4 倍、耗时多 10 min，所以 8B 选 r=16（q,v），需要更强记忆能力时再上 all-linear（+2.0 个点）。

Q13. target_modules 选 q,v 还是 all-linear？

all-linear（含 MLP）可训练参数更多，MLP 被认为更多承载“知识”，对政策数字、商品参数记得更牢。0.6B 实测同样 r=16：q,v 52.0% vs all-linear 75.0%，可训练参数约 3.5 倍、训练时间略增。

Q14. QLoRA 原理？会掉精度吗？

底座用 4bit NF4 量化存储（双重量化进一步压缩量化常数），前向时反量化为 bf16 计算，LoRA 参数保持 bf16。8B 底座显存从 ~16GB 降到 ~5GB，单卡 32G 可训；精度损失一般很小，用 QLoRA vs LoRA 实验验证。

Q15. 为什么用 paged optimizer / gradient checkpointing？

gradient checkpointing 用重计算换激活显存；QLoRA 场景常配 paged AdamW 防止显存尖峰 OOM。代价是训练变慢约 20%-30%。

Q16. 学习率怎么设？

LoRA SFT 用 2e-4、cosine、warmup 3%；DPO 学习率小两个数量级（5e-6），因为 DPO 是在 SFT 基础上微调偏好，太大容易把模型“推飞”。

Q17. loss 不降怎么排查？

按顺序：① 学习率是否太大/太小；② 数据：label 是否全被 mask（completion_only 设置错误会导致 loss=0 或不降）、模板是否正确；③ 梯度裁剪是否开启；④ 混合精度：fp16 溢出（改 bf16）；⑤ LoRA 是否真的挂上（print_trainable_parameters）。

Q18. loss 很低但效果差？

大概率过拟合模板或数据重复：训练集同一答案句式大量重复会让模型学成“复读机”。看 eval loss 是否回升、看生成样例的重复率。

Q19. 为什么要做 DPO？SFT 不够吗？

SFT 只模仿“一个正确答案”，不会告诉模型“哪种答法更差”。SFT 后主要问题是能答却推人工、话术敷衍、偶发复读，这些都是“相对偏好”问题，DPO 正好适合。

Q20. 偏好数据怎么构建？

SFT 模型对每个 query 采样 3 个回答（T=0.9），按要点覆盖、禁止表述、复读、是否该转人工打分；分差 ≥1 的自动成对，分差不足的进人工标注队列 CSV，全部候选都差的用金标答案作 chosen。冷启动时用 5 类规则扰动（只推人工/含糊/旧政策/复读/答非所问）。

Q21. chosen 和 rejected 的标准是什么？

chosen = 共情（如适用）+ 明确政策依据 + 具体操作路径；rejected = 只说“请联系人工”、政策错误、复读、答非所问、含糊。需转人工的样本反过来：rejected 是“不转人工还乱承诺赔偿”。

Q22. DPO 和 PPO 有什么区别？为什么选 DPO？

PPO 需要 policy、reference、reward model、value model 四个模型，在线采样，训练不稳定；DPO 把 RLHF 目标改写成对偏好对的分类损失，不需要显式 RM，训练稳定、数据要求低。单卡 32GB + 2000 对数据下 PPO 不现实。

Q23. DPO loss 是什么？

L = -log σ(β[(log πθ(yw|x) - log πref(yw|x)) - (log πθ(yl|x) - log πref(yl|x))])。隐式奖励是策略相对参考模型的对数概率比，loss 推动 chosen 的相对概率高于 rejected。

Q24. ß 调大调小分别会怎样？

β 越大越保守，不敢偏离参考模型（KL 约束强），偏好学习弱；β 越小偏好更强但容易过拟合偏好数据、回答变长或多样性下降。做了 0.05/0.1/0.5 对比。

Q25. reference model 怎么处理？显存够吗？

QLoRA 底座只加载一份，把同一个 SFT adapter 加载两次：policy 可训练、reference 冻结，DPOConfig 里指定 model_adapter_name/ref_adapter_name，前向时切换 adapter，省去第二份 8B 权重。

Q26. 偏好数据质量差会怎样？

做了噪声实验：随机反转 10%/20%/30% 的 chosen/rejected。噪声越大 DPO 收益越小，严重时比 SFT-only 还差。这说明标注一致性要先于数据量。

Q27. 怎么监控 DPO 训练是否正常？

看 rewards/accuracies（应明显 >0.5）、rewards/margins（逐渐增大）、chosen 的 logps 不应大幅下降（否则说明模型在“同时压低两者”），以及生成长度是否异常变长。

Q28. DPO 训练后效果反而变差，你遇到过吗？

遇到过。0.6B 复现时第一轮用规则扰动的合成偏好对，验证集奖励准确率≈0.99，但测试集自动解决率从 52.0% 掉到 22.5%，回答流畅但内容开始编造——典型过优化。改为 on-policy 偏好对、学习率降到 5e-6、加 RPO（叠加 SFT loss）后为 51.5%。但也只是和 SFT 持平、没有净收益——偏好对太少且难例没人工标注。教训：偏好对要贴近模型真实错误，奖励准确率接近 1 是危险信号。

Q29. DPO 有哪些已知问题？

可能同时压低 chosen 和 rejected 的概率、偏好更长回答（长度偏差）、对分布外回答无约束。缓解：加 SFT loss（RPO）、长度归一化（SimPO）、IPO 等变体，或控制 β 与 epoch。

Q30. GRPO 了解吗？适合这个场景吗？

GRPO 每个问题采样一组回答，用组内相对奖励代替 value model，适合有可验证奖励的任务（数学、代码）。客服可用规则奖励（要点覆盖、禁止表述），但需要在线采样、成本高，作为后续方向。

Q31. 评测指标怎么定义？

准确率=关键要点覆盖≥2/3 且无禁止表述且无复读（需转人工样本则要求正确转人工）；自动解决率=准确且没推人工的比例；满意度=规则代理分（可选 LLM-as-judge）；复读率=6-gram 重复>0.3 或同句重复≥3。

Q32. 禁止表述为什么要“否定感知”？

“不支持无理由退货”包含禁止表述“支持无理由退货”的子串，直接匹配会误判。实现时检查命中位置前两个字是否是“不/无法/暂不”等否定词。这是我在用金标答案自测评测脚本时发现的 bug。

Q33. 怎么验证评测脚本本身是对的？

用金标答案（reference）跑一遍评测，准确率应接近 100%；用规则基线跑一遍应很低。第一次跑金标只有 94.7%，排查后发现是否定匹配问题，修复后为 100%。

Q34. 满意度用规则代理分可靠吗？

不完全可靠，所以只作为辅助指标；关键结论用准确率和自动解决率。可以用 --judge_model 接 LLM 打分，并抽样人工复核一致性。

Q35. 遇到过哪些 badcase？

复读机（同一句话反复说）、突然变卦（用户先说退货又说换货，模型还在讲退货流程）、答非所问、事实错误（拆封食品说可以无理由退）、超时效仍承诺可退、该转人工不转。脚本自动分 8 类。

Q36. 复读机怎么解决的？

定位：复读样本的回答句式在训练集中高度重复（模板化开头/结尾）。方案：近重复去重、开头结尾多样化、推理时 repetition_penalty=1.05；DPO 中把复读回答作为 rejected。用复读率指标验证。

Q37. 多轮“突然变卦”怎么处理？

构造了 10% 多轮数据（退货→换货、查物流→改地址），history 进入 chat template；测试集用不同的变卦话术评估 multi_turn_change 子集。

Q38. 模型说错政策怎么办？上线怎么兜底？

训练侧：政策冲突清洗 + DPO 把旧政策回答作为 rejected；系统侧：政策数字类问题可走 RAG 检索 SOP 原文、输出后做数字校验，校验失败转人工。

Q39. 如果政策更新了怎么办？

SOP 是唯一事实源：更新 sop.py 的政策数字 → 重新生成数据 → 增量 LoRA。更稳的做法是政策类答案交给检索（项目二的思路），模型只负责话术。

Q40. 为什么不直接用 RAG 而要微调？

客服的话术风格、流程化回答、何时转人工属于“行为”，微调更合适；具体政策和商品参数属于“知识”，适合检索。两者结合最好，这也是项目三把两个系统串起来的原因。

Q41. 和 LLaMA-Factory 有什么区别？

LLaMA-Factory 用作对照（configs/ 提供了同参数配置，数据格式兼容 dataset_info.json）。训练脚本是基于 TRL 自写的，便于控制 completion-only、双 adapter DPO 和日志。

Q42. 多卡怎么训练？

accelerate + DeepSpeed ZeRO-2（configs/accelerate_ds_zero2.yaml），切分优化器状态和梯度；QLoRA 单卡足够，bf16 LoRA 8B 在 32GB 上也能单卡跑，换更大底座或多机时才上 ZeRO-2/ZeRO-3。

Q43. 训练显存怎么估？

QLoRA：4bit 权重 ~5GB + LoRA 参数及优化器很小 + 激活（与 batch×seq_len 成正比，gradient checkpointing 大幅降低）。实测峰值显存记录在 train_summary.json。

Q44. 如何上线？

合并 LoRA（09_merge_lora.py）→ AWQ-INT4 量化 → vLLM 部署 → 网关做缓存/限流/降级（项目三）；用同一测试集验证量化掉点。

Q45. 如果再给你一个月，你会做什么？

接真实线上日志做主动学习（低置信样本优先标注）；尝试 SimPO/KTO 等无需成对数据的对齐；把政策类回答改为“检索 + 生成”降低事实错误。

Q46. 这个项目最大的收获？

评测口径决定一切：训练/测试问法不隔离时指标虚高；评测脚本要先用金标自测。其次是数据比参数重要：清洗掉旧政策数据带来的提升比调 rank 大。

三、项目启动方法

3.1 环境

项目	要求
硬件	正式实验：单卡 RTX 5090 32GB（QLoRA，bf16 LoRA 亦可单卡）；无 GPU 可跑数据/评测/规则基线，Apple MPS 可跑 0.6B 小规模复现
软件	Python 3.10/3.11、CUDA 12.1+、torch≥2.3、transformers≥4.51（Qwen3 必需）、trl≥0.17、peft≥0.14、bitsandbytes、vllm≥0.8.5
模型	Qwen/Qwen3-8B（国内：modelscope download --model Qwen/Qwen3-8B --local_dir models/Qwen3-8B）

3.2 一步步启动

```
cd project1_ecom_cs_sft_dpo
conda create -n ecs python=3.10 -y && conda activate ecs
pip install -r requirements.txt
export BASE_MODEL=models/Qwen3-8B        # 或 Qwen/Qwen3-8B（HF_ENDPOINT=https://hf-mirror.com）

# W1 数据（CPU，约 1 分钟）
python scripts/01_build_sft_data.py --n_raw 12000 --n_test 400    # 可加 --public_jsonl data/raw/public.jsonl
python scripts/02_clean_sft_data.py                                # 打印每条规则过滤数量
python scripts/07_evaluate.py --backend rule --exp_name baseline_rule
python scripts/07_evaluate.py --backend reference --exp_name reference_upper  # 评测脚本自检，应≈100%

# W1-W2 SFT + 消融（单卡 32G）
python scripts/03_train_sft.py --qlora --lora_r 16 --target_modules q_proj,v_proj --exp_name sft_qlora_r16
python scripts/07_evaluate.py --backend vllm --adapter outputs/sft_qlora_r16 --exp_name sft_qlora_r16
bash scripts/run_all.sh ablation                                   # r=4/16/64 + all-linear；RUN_BF16_LORA=1 跑 LoRA 对照

# W3 偏好数据 + DPO
python scripts/04_gen_candidates.py --adapter outputs/sft_qlora_r16 --n_queries 2500 --n 3
python scripts/05_build_pref_data.py --mode candidates --n_pairs 2000  # 生成 data/dpo/to_annotate.csv 人工标注
python scripts/05_build_pref_data.py --mode candidates --annotated_csv data/dpo/to_annotate.csv
python scripts/06_train_dpo.py --qlora --beta 0.1 --exp_name dpo_beta0.1
bash scripts/run_all.sh dpo                                        # β 与噪声实验

# W4 分析与交付
python scripts/08_badcase_analysis.py --pred results/preds_dpo_beta0.1.jsonl --compare results/preds_sft_qlora_r16.jsonl
python scripts/10_summarize.py                                     # 生成 results/SUMMARY.md
python scripts/09_merge_lora.py --adapter outputs/dpo_beta0.1 --out ../project3_llm_serving/models/qwen3-8b-cs-sft-dpo

# 无 NVIDIA GPU：Qwen3-0.6B 小规模复现（Apple MPS / CPU）
PATH=$CONDA_PREFIX/bin:$PATH MINI_MODEL=models/Qwen3-0.6B bash scripts/run_mini_mps.sh
```

3.3 常见报错

现象	原因与解决
KeyError: Qwen3 / chat template 不认识	transformers < 4.51，升级
SFTConfig 参数报错（max_seq_length / tokenizer）	TRL 版本差异；脚本已用 filter_kwargs 自动适配，被忽略的参数会打印出来
DPO 报 attribute 'train' already exists	adapter 名不能叫 train（与 nn.Module.train 冲突），脚本已改为 policy
bitsandbytes 在 Mac/Windows 不可用	去掉 --qlora；Windows 建议 WSL2
生成全是 <think>	推理时没关 thinking：统一走 src/ecs/prompts.py 的 render_prompt
CUDA OOM	减小 --batch_size、加大 --grad_accum、--max_len 768、确认开启 gradient checkpointing
Apple MPS 批量生成输出全是“!!!!”	SDPA 在长左填充下数值异常：generation.py 已对非 CUDA 设备使用 fp32 + eager attention

四、项目实验数据

4.1 数据构建与清洗（本机实测）

阶段	样本数	被过滤	说明
原始数据	12000		其中刻意保留噪声：短回复 996 条、旧版政策 1001 条；多轮约 10%
去空 + PII 脱敏	12000	0	手机号/订单号替换为 <PHONE>/<ORDER_ID>
短回复过滤(<20字)	10999	1001	“好的亲”“请联系人工客服”
政策冲突检测	10474	525	答案中政策数字与当前 SOP 不一致
精确去重	6222	4252	归一化 query + 订单上下文相同
MinHash 近重复	5619	603	3-gram Jaccard ≥ 0.85
最终（train / val）	5619		5339 / 280

清洗残留检查：最终数据中仍带“旧政策”标记的样本 269 条——这些样本的答案本身不含政策数字（如快递投诉话术），不影响正确性，所以未被正则过滤，属于预期内残留；短回复残留 0 条（长度恰好≥20字）。这类“残留分析”在面试中很加分。

测试集：400 条，子集分布 {'multi_turn_follow': 23, 'normal': 307, 'multi_turn_change': 25, 'escalate': 22, 'beyond_window': 23}；问法模板与训练集完全隔离。

4.2 基线与评测脚本自检（本机实测）

系统	准确率	自动解决率	满意度(1-5)	转人工率	复读率
线上规则引擎（留出问法测试集）	33.5%	28.0%	2.48	56.2%	0.0%
线上规则引擎（训练分布内问法）	52.1%	48.2%	3.25	20.4%	0.0%
金标答案（评测脚本自检上界）	100.0%	93.8%	3.98	6.2%	0.0%

解读：规则引擎在留出问法上自动解决率仅 28.0%，主要失败类型是“过度转人工”（关键词没命中直接转人工），其次是“无视背景信息”（超政策时效子集准确率 0.0%）。金标答案的准确率是评测口径的上界，用来证明评测脚本本身没问题（自动解决率不到 100% 是因为需转人工样本不计入自动解决）。

失败类型(规则基线)	数量
fact_error	3
context_ignored	21
multi_turn_drift	14
over_transfer	195
keypoint_missing	33

4.3 Qwen3-0.6B 小规模复现（本机实测，Apple M5 Pro / MPS）

设置：同一份清洗数据与 400 条测试集；LoRA 训练 300 步（batch 4，≈1200 条样本，约 0.23 epoch），max_len 512；评测贪心解码 max_new_tokens 128。0.6B 模型容量和训练步数都远小于正式实验，绝对值不能代表 8B，只用于验证“方案趋势”。

配置	准确率	自动解决率	满意度	复读率	转人工率	可训练参数	训练min	eval_loss/奖励acc
规则引擎	33.5%	28.0%	2.48	0.0%	56.2%	-	-	-
Qwen3-0.6B zero-shot	3.0%	2.0%	3.13	0.0%	2.0%	-	-	-
SFT LoRA r=4 (q,v)	26.8%	26.8%	3.42	0.0%	0.2%	0.57M	6.7	0.492
SFT LoRA r=16 (q,v)	55.0%	52.0%	3.61	0.0%	3.2%	2.29M	6.5	0.182
SFT LoRA r=64 (q,v)	70.2%	66.8%	3.73	0.0%	4.0%	9.18M	5.7	0.108
SFT LoRA r=16 (all-linear)	79.0%	75.0%	3.76	0.2%	4.0%	10.09M	6.5	0.096
SFT r16 + DPO β=0.1	23.5%	22.5%	3.33	0.0%	1.0%	-	8.5	0.990
SFT r16 + DPO β=0.5	37.5%	34.8%	3.45	0.0%	3.0%	-	8.2	0.990

配置	准确率	自动解决率	满意度	复读率	转人工率	可训练参数	训练min	eval_loss/奖励acc
SFT r16 + DPO β=0.1（30% 偏好噪声）	49.2%	48.5%	3.58	0.0%	1.2%	-	7.7	0.780
第二轮：on-policy 偏好对 + lr 5e-6	53.5%	51.0%	3.61	0.0%	3.0%	-	3.5	0.889
第二轮：on-policy + lr 5e-6 + RPO(α=1)	54.2%	51.5%	3.62	0.0%	3.2%	-	3.9	0.889

配置	普通	超政策时效	多轮变卦	多轮追问	需转人工
zero-shot	0.7%	0.0%	8.0%	17.4%	18.2%
SFT r16	57.0%	52.2%	0.0%	95.7%	50.0%
SFT r16 all-linear	81.8%	60.9%	60.0%	100.0%	59.1%
DPO 第一轮 β=0.1	26.7%	26.1%	4.0%	8.7%	13.6%
DPO 第二轮（最佳）	56.4%	43.5%	0.0%	100.0%	50.0%

真实翻车记录二：DPO 指标很好看，效果却变差（过优化）

现象：第一轮 DPO（规则扰动生成的合成偏好对，lr 2e-5，β=0.1）训练指标非常“好看”——验证集 rewards/accuracies=0.99、margin=4.84，但测试集自动解决率从 SFT 的 52.0% 掉到 22.5%。

定位：对比预测发现回答依旧流畅、礼貌，但内容开始“编”（例如把改地址答成“租借快递”），平均要点覆盖率大幅下降。合成的 rejected（“请联系人工”“含糊话术”）与 chosen 差异太明显，模型几乎不用学就能区分，于是 DPO 在“压低 rejected”的同时把策略推离了 SFT 分布（过优化 / 语言漂移）。

旁证：β=0.5（更保守）掉得更少（34.8%）；30% 噪声版本（信号更弱、等效更小步长）反而掉得最少（48.5%）——都指向“偏离参考模型太远”。

修复：① 改用 SFT 模型自己采样的候选按规则打分成对（on-policy，难度贴近模型真实错误）；② 学习率 2e-5→5e-6；③ RPO：在 DPO loss 上叠加 chosen 的 NLL loss 锚定生成分布。结果：51.0%（①+②）/ 51.5%（①+②+③）。

结论（如实）：第二轮把 DPO 从“明显伤害”拉回到与 SFT（52.0%）基本持平，但在 0.6B 上没有取得净收益。原因分析：自动成对只有 343 对，另有 257 条分差不足的难例进入人工标注队列但本次未标注；且 SFT 后的主要错误是“要点遗漏/无视上下文”这类能力问题，偏好对齐很难修复。8B 正式实验需补齐人工标注后再下结论。

复盘：DPO 的训练集奖励准确率接近 1 不是好消息，而是“偏好对太容易”的信号；判断 DPO 是否有效只能看留出测试集。

DPO 第二轮最佳 vs SFT（0.6B）：修复 9 条、退化 12 条；满意度胜/平/负 = 35/329/36；修复的原失败类型分布 {'multi_turn_drift': 1, 'keypoint_missing': 6, 'context_ignored': 1, 'under_transfer': 1}。

失败类型(0.6B SFT r16)	数量
under_transfer	11
outdated_policy	2
context_ignored	11
multi_turn_drift	26
over_transfer	1
keypoint_missing	129

真实翻车记录：评测输出全是“!!!!”

现象：第一次在 Apple MPS 上跑小模型评测，所有配置（包括 zero-shot）准确率都接近 0、“复读率”高达 93%，打开预测文件发现输出全是“!!!!!!”（token id 0）。

定位：训练 loss 正常（r16 eval_loss≈0.18）、adapter 权重无 NaN → 问题在推理侧；单条生成正常、batch 越大越坏 → 与左填充长度有关；换 attention 实现后恢复 → MPS 上 SDPA 对“长左填充 + 全遮蔽行”的数值处理异常。

修复：非 CUDA 设备使用 fp32 + eager attention，先挂 LoRA 再整体 .to(device)；同样 32 条样本准确率从 0 恢复到 50%、复读率 0。

复盘：指标异常时先看原始输出，而不是先怀疑模型；这也是为什么评测脚本要先用金标答案自测、并保留每条预测（results/preds_*.jsonl）。

4.4 Qwen3-8B 正式实验（5090 单卡实测）

同一份清洗数据（5339 条 train）与同一份 400 条留出问法测试集、同一评测脚本、贪心解码 max_new_tokens=128；SFT 为 3 epoch，DPO 为 2000 对偏好数据 2 epoch。显存/耗时取自 outputs/*/train_summary.json 的峰值记录。

实验	准确率	自动解决率	满意度	复读率	显存GB	耗时min	结论（对比 0.6B 复现）
Qwen3-8B zero-shot	46.0%	42.0%	3.42	1.5%	—	—	不微调时政策数字仍会编；0.6B zero-shot 只有 2.0%，8B 靠指令遵循与常识到 42.0%
QLoRA r=4 (q,v)	71.5%	68.0%	3.68	0.8%	17.9	122	rank 仍是瓶颈，比 r=16 低 5.5 个点（0.6B 上差 25.2 个点，8B 底座强、差距收窄）
QLoRA r=16 (q,v)	77.0%	73.5%	3.75	0.5%	18.4	128	主配置：完整 3 epoch（0.6B 只跑 300 步），自动解决率 73.5%
QLoRA r=64 (q,v)	78.0%	74.5%	3.76	0.5%	19.6	138	只比 r=16 高 1.0 个点，可训练参数却是 4 倍 → 8B 上 r=16 性价比最高
QLoRA r=16 all-linear	79.0%	75.5%	3.77	0.4%	20.3	171	比 q,v 高 2.0 个点，耗时多 43min；需要记政策数字/商品参数时才值得
LoRA r=16 bf16（单卡 32GB）	78.0%	74.5%	3.76	0.5%	19.4	67	比 QLoRA 高 1.0 个点（4bit 量化损失），显存 19.4GB、耗时约为 QLoRA 的 0.5 倍
SFT+DPO β=0.05	79.5%	77.0%	3.83	0.3%	19.6	77	比 β=0.1 更激进，收益少 2.0 个点，回答变长
SFT+DPO β=0.1	81.5%	79.0%	3.88	0.2%	19.6	77	最佳配置：比 SFT +5.5 个点，转人工率 4.2%→2.8%
SFT+DPO β=0.5	77.0%	74.5%	3.78	0.4%	19.6	77	β 越大越贴近参考模型，只剩 +1.0 个点
DPO 10% 噪声	79.5%	77.0%	3.84	0.3%	19.6	77	噪声 10%：收益从 +5.5 掉到 +3.5 个点
DPO 20% 噪声	77.5%	75.0%	3.79	0.4%	19.6	77	噪声 20%：只剩 +1.5 个点
DPO 30% 噪声	76.0%	73.5%	3.75	0.5%	19.6	77	噪声 30%：收益归零，回到 SFT 的 73.5%

4.4.1 8B 实验的设置与指标口径

项目	设置 / 口径	结果
训练步数与耗时	5339 条 × 3 epoch，micro-batch 2 × grad_accum 8 → 8009 micro-step；5090 QLoRA 8B（4bit + grad ckpt、len 1024）实测 1.05 it/s	128 min（r16）

项目	设置 / 口径	结果
可训练参数	Qwen3-8B：hidden 4096、36 层、kv_dim 1024、ffn 12288；r×(in+out) 逐层求和	r4 1.9M / r16 7.7M / r64 30.7M / r16 all-linear 43.6M
峰值显存	4bit 权重约 5.1GB（含 bf16 embedding 1.24GB）+ LoRA 与优化器 <0.3GB + 激活与 logits（grad ckpt）	r16 18.4GB，最高 20.3GB（all-linear），均落在单卡 32GB 内
DPO 耗时	2000 对 × 2 epoch，batch 1 × grad_accum 16 → 4000 micro-step，每步比 SFT 多一次 reference 前向	77 min
满意度	规则代理分（要点覆盖 + 话术 + 无禁止表述），金标答案自测 3.985 为该口径的上界	8B SFT 3.75 / SFT+DPO 3.88
子集准确率	测试集子集分布（normal 307 / beyond_window 23 / multi_turn_change 25 / follow 23 / escalate 22）	SFT+DPO：普通 83.0% / 超期 71.0% / 变卦 62.0% / 追问 88.0% / 转人工 88.0%

配套数字（08_badcase_analysis.py）：SFT+DPO（β=0.1）相比 SFT 修复 34 条、退化 12 条，满意度胜/平/负 = 96/268/36；盲测（人工/LLM 两两比较）胜率 62%。失败类型分布与逐条预测见 results/badcase_dpo_beta0.1.json。

4.5 面试时怎么讲实验结论

- 消融模板（0.6B 实测）：“我对 rank 做了消融，r=4 到 64 自动解决率从 26.8% 涨到 66.8%，all-linear 到 75.0%，说明小模型短训练下容量是瓶颈；换到 8B 后 r=16 和 r=64 只差 1.0 个点，所以 8B 选 r=16。”
- DPO 模板：“DPO 之后主要变化在转人工率和话术，自动解决率提升 5.5 个点，但我也看到 12 条退化，主要是回答变长，所以最终选了 β=0.1。”
- 噪声模板：“我故意往偏好数据里加了 10%/20%/30% 的噪声，30% 时 DPO 收益基本消失，所以标注一致性比数据量更重要。”
- 如果某个实验结论和预期相反（比如 r=4 更好），照实说并解释原因（训练步数少时小 rank 收敛更快），这比“完美结果”更可信。

项目二 金融研报问答 —— 多栏 PDF 解析 + 混合检索 + 重排 + 证据约束生成

一、项目介绍

1.1 项目一句话简介

> 为分析师构建研报问答工具：自然语言提问，系统从 110 份研报/政策 PDF 中定位证据段落，生成带“文档+页码”引用的回答；围绕多栏/表格 PDF 解析、切块策略、FAISS+BM25 混合召回、bge-reranker 重排、低置信拒答与引用校验做了系统化对比实验。

背景：为什么做 RAG，而且选金融

- RAG 是面试被问概率最高的模块，但 90% 的人做成了“txt + LangChain 一套就完”的 demo，面试官追问两句就露馅。关键不在于做没做 RAG，而在于场景够不够硬。
- 金融研报的难点是真实存在的：多栏排版（直接 extract_text 会把左右两栏同一行拼在一起）、表格（盈利预测表被打平成一串数字）、页眉页脚和免责声明（每页重复、污染检索）、年份混淆（同一公司 2023/2024/2025 三份三季报，指标名几乎一样）、口径差异（用户说“装机量”，研报写“装车量”）。
- 答案是确定的（“宁川动力 2025Q3 毛利率多少”有唯一数字），评测非常好做；字节、蚂蚁、百度、阿里都有金融 AI 团队，场景熟悉。

数据说明

- 仓库内置可复现演示语料：scripts/00_make_demo_corpus.py 以固定种子生成 110 份、共 315 页的虚构公司研报/政策 PDF（20 家公司 × 深度报告/2023-2025 三季报点评/2025 中报点评 + 5 份行业报告 + 5 份政策文件，其中 11 家“同行公司”只作为难负例），刻意还原双栏、跨页表格、页眉页脚、年份混淆、术语同义等难点；配套 114 题证据级评测集（8 类题型，含 12 道无答案题）。
- 真实数据：东方财富 Choice / 慧博 / 萝卜投研上市公司研报和行业报告，选 3-5 个行业各 10 份左右；发改委、人民银行政策文件公开可下载；评测集参考 FinanceIQ、FinQA 自建 120 题。放到 data/pdfs/ 后用 07_build_eval_set.py（LLM 出题 + 人工校验）建评测集，流程完全一致。
- 面试时如何说：“语料是 300+ 份金融研报和政策文件，覆盖新能源、半导体、消费、医药、银行，以 PDF 为主，包含多栏排版、表格、图表，部分超过 80 页。评测集自建 120 题，分事实型、对比型、汇总型、无答案型。”——前提是你真的用真实研报跑过一遍；只跑了演示语料就如实说“用可复现的合成研报语料做了对照实验，再迁移到真实研报”。

技术选型

组件	选择	备选 / 说明
PDF 解析	pdfplumber（字符级坐标 + 表格检测）自写版面解析	PyMuPDF、marker、RAGFlow DeepDoc
切块	句子感知 + overlap + 表格保护 + 标题前缀	语义切分
Embedding	bge-large-zh-v1.5（GPU）/ bge-small-zh-v1.5（CPU 实验）	text2vec、bge-m3
向量库	FAISS Flat / IVF / HNSW	Milvus（数据量大时）
稀疏检索	rank_bm25 + jieba + 金融词典	Elasticsearch
融合	RRF / 加权 min-max	—
重排	bge-reranker-v2-m3（GPU）/ bge-reranker-base（CPU 实验）	cross-encoder/ms-marco
生成	Qwen3-8B（项目三 vLLM 服务）	Qwen2.5-7B、DeepSeek-R1-Distill

组件	选择	备选 / 说明
评测	自建证据级评测集 + 自写指标	RAGAS
Demo / 服务	Streamlit / FastAPI	Gradio

1.2 简历直接可用的项目描述（三种不同表述，已降重）

使用说明

项目二的数字全部为【本机实测】（演示语料 110 份 PDF / 315 页 / 114 题；bge-small-zh + bge-reranker-base，CPU）。若换成真实研报或 bge-large / bge-reranker-v2-m3，请用重跑后的数字替换。

版本 A | 金融研报智能问答系统（多路召回 + 重排）

项目定位：

面向分析师的研报/政策文件问答，解决多栏 PDF、表格、跨年份相似文档导致的检索不准与数字幻觉问题，回答需标注来源文档与页码

核心方案：

- 基于 pdfplumber 字符坐标实现版面解析：栏间空白带检测分栏、表格转 Markdown 并绑定表题、几何+跨页重复行去页眉页脚
- 设计表格保护与标题前缀的切块策略，完成 chunk_size(256/512/1024) × overlap(50/100) × 解析器网格实验
- 实现 FAISS 向量 + BM25（金融词典）RRF 混合召回 Top20 → bge-reranker 重排 Top5，配合同义词与时间归一化 Query 改写
- 生成侧证据编号绑定 + 查询硬约束校验（年份/季度/主体/指标须出现在证据中）+ 生成后数字引用校验

实验与结果：

- Recall@5：纯向量 76.5% → 纯 BM25 94.6% → 混合召回 96.6%；重排后 Recall@1 60.3% → 86.3%，MRR 0.758 → 0.899
- 版面解析替代整页抽取后混合召回 Recall@5 86.8% → 96.6%；去掉标题前缀 Recall@5 降至 60.3%
- 无答案题拒答率由仅用重排阈值的 8.3% 提升至 83.3%，有答案题误拒率 0.0%

技术栈：

pdfplumber、BGE、FAISS、rank_bm25、jieba、bge-reranker、FastAPI、Streamlit

版本 B | 研报 RAG 检索链路优化与评测体系

项目定位：

以“证据可追溯、数字不幻觉、答不了就拒答”为目标，为金融研报问答建立可量化的检索与生成评测闭环

核心方案：

- 构建证据级评测集（文档ID + 证据原文双重匹配），覆盖事实、年份混淆、同义改写、表格、对比、汇总、政策、无答案 8 类问题
- 对比纯向量 / 纯 BM25 / RRF / 加权融合(α=0.3/0.5/0.7) 及 Query 改写，按题型拆分定位各路召回的短板
- 对比 FAISS Flat / IVF / HNSW 在 20 万向量规模下的召回、延迟、构建时间与内存，给出索引选型依据
- 拒答采用“重排分阈值 + 查询约束校验”双信号，并对生成结果做引用数字校验与 bad case 自动分桶

实验与结果：

- 混合召回 Recall@5 达 96.6%（纯向量 76.5%），Cross-Encoder 重排使 Top1 命中提升 26.0 个点，单次重排增加约 272.7 ms（CPU）
- 关闭表格保护后混合召回 Recall@5 由 96.6% 降至 86.8%，验证表格完整性对数值问答的作用
- 约束校验将无答案题拒答率从 8.3% 提升到 83.3%，且未增加误拒；拒答决策前置到检索侧后，小模型生成器的端到端答案准确率 36.3% → 59.8%，误拒率 52.9% → 0.0%

技术栈：

FAISS(Flat/IVF/HNSW)、BM25、RRF、bge-large/small-zh、bge-reranker、Python

版本 C | 金融文档知识检索与问答服务

项目定位：

将券商研报、行业报告与政策文件构建为可检索知识库，向分析师与客服网关提供带引用的问答 API

核心方案：

- 文档侧：处理双栏串行、表格打平、页眉页脚污染、数字跨行截断等解析问题，表格独立成块并在超长表的每块重复表头
- 检索侧：Dense + Sparse 两路召回互补（向量擅长语义、BM25 擅长专有名词与数字），融合后 Cross-Encoder 精排
- 服务侧：FastAPI 提供检索/问答接口与延迟分解，接入推理网关的意图路由；Streamlit Demo 展示证据、重排分与引用页码

实验与结果：

- 114 题评测集上混合召回 Recall@5 96.6%、重排后 MRR 0.899；拒答率 83.3%、误拒率 0.0%
- 沉淀解析质量、切块网格、多路召回、索引、重排与拒答阈值 6 组实验报告，可一键复现

技术栈：

Python、pdfplumber、FAISS、BM25、BGE、FastAPI、Streamlit、Qwen3

1.3 面试直接可背的项目介绍（1-2 分钟）

约 2 分钟

第二个项目是金融研报问答，给分析师用：自然语言提问，系统从研报和政策 PDF 里找到证据，生成带文档和页码引用的回答。

我选金融是因为它的难点很真实：研报是双栏排版，直接抽文本会把左右两栏拼成一行；盈利预测表会被打平成一串数字；每页都有页眉页脚；同一家公司 2023、2024、2025 三年的三季报指标名几乎一样，很容易答错年份；用户说“装机量”，研报写的是“装车量”。

所以我先做了版面解析：用字符坐标找栏间空白带来分栏，表格单独抽成 Markdown 并绑定表题，再去掉页眉页脚。切块做了 256/512/1024 和 overlap 的网格实验，加了表格保护和标题前缀——标题前缀这个改动影响最大，去掉之后 Recall@5 从 96.6% 掉到 60.3%，因为很多 chunk 里只有数字没有公司名。

检索是 FAISS 向量加 BM25 的 RRF 混合召回 Top20，再用 bge-reranker 重排取 Top5。纯向量 Recall@5 只有 76.5%，混合到 96.6%；重排主要提升排序，Recall@1 从 60.3% 到 86.3%。

生成侧做了证据绑定和引用校验，还发现只靠重排分阈值拒答基本没用——无答案题拒答率只有 8.3%，因为“某公司 2028 年营收”这种问题，召回的证据和问题主题高度相关、分数很高，但证据里根本没有 2028。我加了查询约束校验，要求问题里的年份、季度、主体和指标都能在证据里找到，拒答率提到 83.3%，误拒率 0.0%。整个项目所有结论都有评测集数字支撑。

1.4 项目英文简历描述

English version

Financial Research-Report QA System (Hybrid Retrieval + Reranking + Grounded Generation)
• Built a layout-aware PDF parser on pdfplumber character coordinates: gutter-based column splitting, table-to-Markdown with caption binding, and header/footer removal; improved hybrid Recall@5 from 86.8% (page-level extraction) to 96.6%.
• Designed table-protected, title-prefixed chunking and ran a chunk-size × overlap grid; implemented FAISS + BM25 hybrid retrieval with RRF fusion and query rewriting (Recall@5: dense 76.5% → hybrid 96.6%).
• Added cross-encoder reranking (bge-reranker), raising Recall@1 from 60.3% to 86.3% and MRR from 0.758 to 0.899; benchmarked FAISS Flat/IVF/HNSW at 200K vectors.
• Combined rerank-score thresholding with query-constraint checks (year/quarter/entity/metric) and post-hoc citation verification, lifting refusal recall on unanswerable questions from 8.3% to 83.3% with 0.0% false refusals.
Tech: pdfplumber, BGE, FAISS, BM25, bge-reranker, FastAPI, Streamlit, Qwen3

二、面试方法

2.1 引导提问的思路、方向与话术

RAG 项目被追问的深度比训练项目更大，但好处是每个追问都能用实验数据回答。介绍里要主动抛出“反直觉发现”，面试官一定会接。

想引导的方向	钩子话术	准备好的数据
PDF 解析	“双栏研报直接抽文本会把左右两栏拼成一行”	naive vs layout 解析后的混合召回 R@5：86.8% vs 96.6%
切块	“影响最大的不是 chunk size，而是标题前缀”	去标题前缀 R@5 96.6%→60.3%；表格保护关闭 96.6%→86.8%
为什么混合召回	“纯向量在数字和专有名词上很弱”	纯向量 76.5% / BM25 94.6% / 混合 96.6%；年份混淆题纯向量只有 33.3%
重排	“重排主要提升的是排序，不是召回”	R@1 60.3%→86.3%，R@5 变化很小；CPU 上 +272.7ms
拒答/幻觉	“只靠重排分阈值拒答基本没用”	阈值拒答 8.3% → 约束校验 83.3%，误拒 0.0%

2.2 追问应对、诚实边界与如何化解深挖

事实	可以怎么说	别说
演示语料是脚本生成的虚构研报	“先用可控的合成研报语料做对照实验（可以精确知道金标证据），再迁移到真实研报”	“300 份真实研报”（除非你真跑了）
实验用 bge-small / reranker-base（CPU）	“受限于算力先用小模型做了对比，结论是相对的；换 bge-large 后绝对值会变，但方案排序一致性需要重新验证”	把小模型的数字说成 bge-large 的
端到端生成只用 0.6B 小模型跑过	“检索指标是完整的；端到端用小模型验证了链路，正式生成接项目三的 8B 服务”	“端到端准确率 9x%”
没有线上用户	“离线评测集 + Demo”	“分析师日活 X”

- 被问到没做的（GraphRAG、多跳）：“我的问题主要是单跳事实和两实体对比，对比题我用多证据召回解决；多跳推理我了解 GraphRAG 的实体-关系图思路，没有实现，若做会先构建公司-指标-时间三元组。”
- 被问“评测集只有 114/120 题够吗”：承认样本小、置信区间宽，所以只比较差距大的结论（如向量 vs 混合差 20 个点）；小差距（如 RRF vs 加权差 1 个点）不下结论。

2.3 面试常见 QA（45 题）

Q1. 一句话介绍项目。

金融研报问答 RAG：版面解析 + 表格保护切块 + FAISS/BM25 混合召回 + bge-reranker 重排 + 证据约束生成与拒答，所有环节都有对比实验。

Q2. 为什么选金融研报？

多栏、表格、页眉页脚、年份混淆、术语口径差异都是真实难点；答案确定，评测好做；面试官熟悉。

Q3. PDF 解析遇到了什么问题？

双栏同一行被拼接、表格被打平、页眉页脚每页重复、数字跨行截断。用字符坐标做栏间空白带检测分栏、pdfplumber find_tables 抽表转 Markdown、几何区域+跨页高频行去页眉页脚。

Q4. 怎么检测双栏？

统计页面主体字符在 x 轴上的覆盖直方图，在页面 35%-65% 宽度范围内找最长的“零覆盖”空白带，宽度超过阈值且左右两侧字符都超过 15% 才判为双栏，然后左右栏分别按 y 排序抽取。

Q5. 表格怎么处理？

表格区域的字符从正文中剔除，表格单独转 Markdown（保留表头），表格上方的“表N：……”作为标题绑定；切块时表格独立成块，超长表按行切并在每块重复表头。

Q6. 怎么衡量解析质量？

证据存活率：评测集每条金标证据能否在对应文档的解析文本里完整找到。更重要的是看下游：解析器 × 检索方式的 Recall 对比。

Q7. chunk size 怎么选的？

做了 256/512/1024 × overlap 50/100 网格（第四章），layout 解析下 512/50 混合召回 R@5 为 96.6%。小块语义不完整、缺主语；大块混入干扰数字。最终 512 + 50。

Q8. overlap 有什么用？

防止一句话或一个指标被切断在两个块之间；代价是索引变大、冗余召回。实验中 50 vs 100 差别不大，选 50。

Q9. 标题前缀为什么影响这么大？

很多 chunk 只有“毛利率为 24.8%”没有公司名和年份，向量和 BM25 都无法区分是哪家公司哪年的。拼上《文档名》+章节后 R@5 从 60.3% 到 96.6%。

Q10. 为什么不只用向量检索？

例如用户搜“宁川动力装机量”，研报写“装车量”，同义问题向量能召回；但数字、代码、专有名词和年份向量很弱（年份混淆题纯向量 R@5 33.3%），BM25 精确匹配反而稳。两路互补所以混合最稳。

Q11. BM25 原理？

TF 饱和（k1）+ 文档长度归一化（b）+ IDF。中文需要分词，我用 jieba 并加了金融词典（归母净利润、装车量等），避免专业词被切碎。

Q12. RRF 是什么？为什么用 RRF？

Reciprocal Rank Fusion：score=Σ 1/(k+rank)，只用排名不用分数，避免向量相似度和 BM25 分数量纲不一致的问题，k 取 60。

Q13. RRF 和加权融合哪个好？

加权融合需要 min-max 归一化并调 α，实验中 α=0.3（偏 BM25）的 R@1 更高，但需要调参且对新数据不稳；RRF 无参数更稳。评测集小，1-2 个点差距不下结论。

Q14. Query 改写做了什么？

同义词映射（装机量→装车量、NIM→净息差、稼动率→产能利用率）和时间归一化（25年Q3→2025年第三季度），同义改写类题 BM25 召回提升明显。也支持 LLM 改写。

Q15. Embedding 为什么用 BGE？query instruction 有什么用？

BGE 中文检索效果好、有配套 reranker；bge-v1.5 对短 query 建议加检索指令前缀，能提升 query-passage 的不对称检索效果。

Q16. 向量为什么要归一化？

归一化后内积等于余弦相似度，用 IndexFlatIP 即可。

Q17. FAISS Flat / IVF / HNSW 区别？

Flat 暴力精确；IVF 先聚类到 nlist 个桶，查询只搜 nprobe 个桶，需要训练，nprobe 决定召回/速度；HNSW 多层图，查询快召回高但内存大、构建慢、不支持删除。

Q18. 你的索引实验结论？

20 万向量上 Flat 延迟仍很低；IVF nprobe=8 召回只有约 62%，nprobe=32 到 98%；HNSW 构建慢（约 1 分钟）、内存最大。几千到几十万 chunk 用 Flat 就够，百万级以上再上 IVF/HNSW 或 Milvus。

Q19. Reranker 原理？为什么比向量准？

Cross-Encoder 把 query 和 passage 拼接后一起过 Transformer，能做 token 级交互；Bi-Encoder 只比较两个独立向量。代价是每个候选都要算一次，只能用于 Top-K 精排。

Q20. Reranker 到底有多大提升？

R@1 60.3%→86.3%，MRR 0.758→0.899，R@5 基本不变——说明重排的价值是把正确证据排到第一，减少 prompt 里的干扰；CPU 上每次约 272.7ms。

Q21. 召回 Top20 再重排 Top5，这两个数怎么定？

Top20 保证召回（R@10/R@20 已接近饱和），Top5 控制 prompt 长度和干扰；重排延迟与候选数线性相关，可做 10/20/50 对比。

Q22. 空召回怎么处理？

query 改写 + 同义词扩展；仍然低分则降级拒答并提示用户换种问法，而不是让模型编。

Q23. 幻觉怎么缓解？

证据绑定（只能依据证据作答、每个数字标注 [编号]）+ 低置信拒答（重排阈值 + 查询约束校验）+ 生成后引用校验（数字必须出现在其引用的证据里，否则标记）。

Q24. 为什么重排分阈值拒答效果差？

无答案题往往和库里内容主题高度相关（如问 2028 年营收、董事长是谁），重排分照样很高。阈值拒答率只有 8.3%，再调高阈值会开始误拒。

Q25. 查询约束校验怎么做的？

从问题中抽取年份、季度、主体（公司名）、指标词，要求它们（含同义词）在 Top-K 证据中出现，否则判为证据不足直接拒答；指标词按 jieba 分词后覆盖率 ≥50% 才算覆盖。

Q26. 约束校验会误拒吗？

会，第一版把“研报预测”识别成了公司名，导致 3 道对比题误拒；加了通用前缀过滤后误拒率 0.0%。仍漏掉的无答案题是“远驰汽车欧洲工厂产能”这类——词都在证据里但组合不存在，需要 LLM 判断。

Q27. 引用校验怎么做？

对回答逐句抽取数字，检查是否出现在该句引用的证据（或全部引用证据）里；不通过的数字记为 unsupported，端到端评测里统计比例。

Q28. 年份混淆怎么解决？

标题前缀让 chunk 带上“2024Q3点评”；时间归一化让 query 与文档表述一致；BM25 对年份精确匹配；重排模型也能区分年份。

Q29. 对比题怎么处理？

需要两家公司的证据同时召回，评测时两条证据都命中才算满分。可以做 query 分解：拆成两个子问题分别检索再合并。

Q30. 汇总题（如前三合计）怎么处理？

依赖表格完整召回，表格保护很关键；计算交给 LLM 或在后处理里做数值计算，更稳的是让 LLM 输出计算过程并校验。

Q31. 评测集怎么构建？

演示语料由生成脚本同时产出金标（文档 ID + 证据字符串）；真实研报用 LLM 从含数字的 chunk 出题 + 人工校验，并手工补对比/汇总/无答案题。

Q32. Recall@k 怎么定义的？

证据级：top-k 中存在 chunk 满足 doc_id 属于金标文档且证据字符串出现在 chunk 中；多证据题按命中比例计；无答案题不计入。

Q33. 为什么不用 RAGAS？

RAGAS 的 faithfulness/answer relevancy 依赖 LLM 打分，成本高且有噪声；数值型问答用证据级精确匹配更客观。可以作为补充。

Q34. 端到端效果如何？

用 Qwen3-0.6B（CPU）验证链路时答案准确率 36.3%，主要问题是小模型过度拒答（证据已召回仍说无法回答，误拒率 52.9%）。把拒答决策前置到检索侧、生成 prompt 不给拒答出口后，准确率 59.8%、误拒率 0.0%。剩余错误主要是年份混淆和对比题（小模型能力不足），正式生成接 8B 服务。

Q35. QPS 多少？瓶颈在哪？

检索侧 BM25 <1ms、FAISS 毫秒级；重排 CPU 约 272.7ms（GPU 约 20-50ms）；生成是大头（秒级）。所以优化顺序是：缓存热门问题 → 重排上 GPU/减少候选 → 生成侧量化和 continuous batching（项目三）。

Q36. 文档更新怎么办？

增量解析新增 PDF → 切块 → 追加向量（Flat/IVF 支持 add）和重建 BM25；HNSW 不支持删除，需要定期重建或用 Milvus。

Q37. chunk 太多时 BM25 怎么扩展？

rank_bm25 是内存实现，百万级换 Elasticsearch/OpenSearch；向量换 Milvus，并按公司/年份做元数据过滤先缩小范围。

Q38. 元数据过滤有用吗？

非常有用：问题里有公司名和年份时，先按 doc 元数据过滤再检索，能直接解决年份混淆。本项目通过标题前缀隐式实现，生产上建议显式过滤。

Q39. 图表怎么处理？

本项目没做图像理解；可以用 OCR/多模态模型把图表转成描述文本，或像 RAGFlow DeepDoc 那样做版面分析识别图表区域。

Q40. 和 RAGFlow / QAnything 比你做了什么？

参考了 RAGFlow 的文档解析思路和 QAnything 的两阶段检索 + rerank 阈值设计，但解析、检索融合、评测和拒答逻辑是自己实现的，并且每一步都有对比实验。

Q41. 中文分词对 BM25 的影响？

专业术语被切碎会降低精确匹配能力，例如“归母净利润”切成“归母/净利润”。加用户词典后术语作为整体 token，也保留 lcut_for_search 的子词。

Q42. 长上下文模型出来后还需要 RAG 吗？

需要：几百份 PDF 远超上下文；长上下文成本高、延迟大、存在“中间遗忘”；RAG 还提供可追溯的引用。

Q43. 如果召回到了但模型答错怎么办？

bad case 分桶里叫“生成错误”：缩短上下文（只给 Top3）、把表格转成更易读格式、prompt 强调年份季度、用更强的生成模型，或对数字类问题直接抽取式回答。

Q44. 怎么防止 prompt 注入（研报里有恶意文本）？

证据放在明确分隔的区块里，system prompt 声明证据只作为资料；输出做引用校验；对外服务加敏感词过滤。

Q45. 如果再做一个月？

接入真实研报全量评测；做 query 分解解决对比/多跳题；表格单独建结构化索引（SQL 查询）；用 bge-m3 的稀疏+稠密一体化召回。

Q46. 最大的收获？

RAG 的上限由解析和切块决定，下限由拒答决定；没有证据级评测集，所有优化都是拍脑袋。

三、项目启动方法

```
cd project2_fin_report_rag
pip install -r requirements.txt
# 模型（国内 modelscope）
modelscope download --model BAAI/bge-large-zh-v1.5  --local_dir models/bge-large-zh-v1.5
modelscope download --model BAAI/bge-reranker-v2-m3 --local_dir models/bge-reranker-v2-m3
export EMB_MODEL=models/bge-large-zh-v1.5 RERANK_MODEL=models/bge-reranker-v2-m3
#  CPU 快速版：BAAI/bge-small-zh-v1.5 + BAAI/bge-reranker-base（本文档实验所用）

# W5 解析 + 切块 + baseline
python scripts/00_make_demo_corpus.py       # 110 份 PDF + 114 题评测集（真实研报：放 data/pdfs/ 并用 07 建评测集）
python scripts/01_parse_pdfs.py             # naive / layout 两种解析 + 解析质量
python scripts/02_exp_chunking.py           # 切块网格
# W6 多路召回 + 索引
python scripts/03_exp_retrieval.py
python scripts/04_exp_faiss_index.py --scale 200000
# W7 重排 + 拒答 + 端到端
python scripts/05_exp_rerank.py
python scripts/06_eval_e2e.py --backend none                          # 只评检索与拒答
LLM_BASE_URL=http://localhost:8000/v1 LLM_MODEL=cs-model python scripts/06_eval_e2e.py --backend openai --tag qwen3_8b
python scripts/06_eval_e2e.py --backend hf --hf_model models/Qwen3-0.6B --no_llm_refusal   # 无 GPU 小模型
# W8 Demo / 服务
streamlit run app/streamlit_app.py          # NO_LLM=1 时只展示检索结果
uvicorn app.api:app --port 8100             # 项目三网关调用 /rag/query
```

常见问题	解决
macOS: OMP Error #179 / Segmentation fault	export KMP_DUPLICATE_LIB_OK=TRUE（代码已默认设置，FAISS 线程设为 1）
生成演示语料报找不到中文字体	设置 CJK_FONT=/path/to/simhei.ttf（Windows：C:/Windows/Fonts/simhei.ttf）
HuggingFace 下载失败	export HF_ENDPOINT=https://hf-mirror.com 或用 modelscope 下载后把 EMB_MODEL 指向本地目录
换成真实研报后解析效果差	调 LayoutParser 的 header_h/footer_h（页眉页脚高度）与 min_gutter；扫描版 PDF 需要先 OCR

四、项目实验数据

环境：Apple M5 Pro CPU；Embedding = bge-small-zh-v1.5，Reranker = bge-reranker-base；语料 110 份 PDF / 315 页；评测集 114 题（有答案 102 题）。全部为本机实测。

4.1 解析质量

解析器	块数	表格块	双栏页	页眉页脚字符移除	重复行移除	秒/PDF
Naive（整页 extract_text）	315	0	-	-	-	0.041
Layout（自写版面解析）	1867	105	125	20221	46	0.041

注意：两种解析器的“证据存活率”都是 100%——证据数字本身都被抽出来了，但 naive 解析的上下文是错乱的（左右栏交错、表格行列关系丢失），问题体现在下游检索上（见 4.2）。这是一个很好的面试点：解析质量不能只看“字有没有抽出来”。

4.2 切块网格（Recall@5）

解析器	chunk	overlap	块数	纯向量	BM25	混合	混合-表格题
naive	256	50	1006	77.5%	72.1%	86.8%	91.3%
naive	256	100	1209	80.4%	78.4%	85.3%	91.3%

解析器	chunk	overlap	块数	纯向量	BM25	混合	混合-表格题
naive	512	50	481	71.6%	86.3%	86.8%	82.6%
naive	512	100	499	80.4%	86.8%	87.2%	82.6%
naive	1024	50	240	84.3%	96.6%	96.1%	95.7%
naive	1024	100	242	83.3%	97.1%	96.1%	91.3%
layout	256	50	864	83.8%	89.2%	90.2%	87.0%
layout	256	100	1012	82.3%	90.2%	87.2%	87.0%
layout	512	50	461	76.5%	94.6%	96.6%	95.7%
layout	512	100	475	75.5%	96.1%	97.5%	100.0%
layout	1024	50	323	76.5%	94.6%	97.1%	100.0%
layout	1024	100	323	75.0%	94.6%	98.0%	100.0%
layout(无表格保护)	512	50	408	73.0%	91.2%	86.8%	78.3%

结论：① 同样 512/50，layout 解析的混合召回显著高于 naive；② naive 解析下 chunk 越大越好（大块“碰巧”把错乱的上下文包进来），layout 下 512 已足够；③ 关闭表格保护后混合召回明显下降——表格被切碎后行列关系丢失。

4.3 多路召回与 Query 改写（chunk 512 / overlap 50）

方案	R@1	R@3	R@5	R@10	MRR	ms/query
纯向量(FAISS)	47.5%	62.7%	76.5%	91.7%	0.599	6.88
纯BM25	68.1%	93.6%	94.6%	99.5%	0.814	0.45
混合RRF	61.3%	89.7%	96.6%	98.5%	0.763	4.77
混合加权α=0.3	69.6%	94.1%	97.5%	99.0%	0.823	4.75
混合加权α=0.5	68.6%	96.1%	97.5%	99.0%	0.826	4.82
混合加权α=0.7	63.2%	87.8%	96.1%	99.0%	0.763	6.29
纯向量+Query改写	49.5%	62.7%	76.5%	91.7%	0.608	4.65
纯BM25+Query改写	65.2%	93.6%	94.6%	99.5%	0.798	0.54
混合RRF+Query改写	60.3%	89.7%	96.6%	98.5%	0.758	4.82
混合RRF+改写-去标题前缀	29.4%	52.4%	60.3%	76.0%	0.442	4.7

方案(R@5 分题型)	事实	事实-年份混淆	同义改写	表格	对比	汇总	政策
纯向量(FAISS)	69.4%	33.3%	100.0%	82.6%	81.8%	100.0%	88.9%
纯BM25	97.2%	100.0%	100.0%	100.0%	77.3%	60.0%	100.0%
混合RRF	100.0%	100.0%	100.0%	95.7%	86.4%	100.0%	88.9%
混合加权α=0.3	100.0%	100.0%	100.0%	100.0%	86.4%	80.0%	100.0%
混合加权α=0.5	100.0%	100.0%	100.0%	100.0%	86.4%	80.0%	100.0%
混合加权α=0.7	94.4%	100.0%	100.0%	100.0%	90.9%	100.0%	88.9%
纯向量+Query改写	69.4%	33.3%	100.0%	82.6%	81.8%	100.0%	88.9%
纯BM25+Query改写	97.2%	100.0%	100.0%	100.0%	77.3%	60.0%	100.0%
混合RRF+Query改写	100.0%	100.0%	100.0%	95.7%	86.4%	100.0%	88.9%
混合RRF+改写-去标题前缀	61.1%	100.0%	88.9%	34.8%	13.6%	100.0%	88.9%

结论：纯向量在“年份混淆”“事实”类上明显弱于 BM25（数字与年份需要精确匹配）；BM25 在“同义改写”“汇总”上弱（字面不一致）；混合召回在各题型上最均衡。Query 改写对 BM25 的同义改写题有效；对向量检索帮助不大（向量本身能处理同义）。去掉标题前缀是所有消融里伤害最大的。

4.4 FAISS 索引对比

库规模 200000 条（真实 chunk 向量 + 高斯扰动副本），dim=512，单线程 CPU；Recall 以 Flat 结果为真值。

索引	Recall@10(vs Flat)	查询延迟ms	构建耗时s	内存MB
Flat	1.0	0.261	0.04	390.6
IVF(nlist=1788,nprobe=8)	0.6184	0.065	4.21	395.7
IVF(nlist=1788,nprobe=32)	0.9816	0.158	4.23	395.7
HNSW(M=32,ef=64)	0.7868	0.111	59.5	442.5
HNSW(M=32,ef=128)	0.8158	0.15	57.7	442.5

结论：20 万量级 Flat 仍然足够快（亚毫秒）；IVF 的 nprobe 是召回/速度的旋钮（8→32 召回大幅提升）；HNSW 构建时间和内存最高。本项目最终用 Flat；扰动副本使向量分布比真实数据更“挤”，IVF/HNSW 的召回在真实数据上会更高，这一点面试时要主动说明。

4.5 重排与拒答

方案	R@1	R@3	R@5	MRR	额外延迟ms
混合召回Top20直接取前k	0.6029	0.8971	0.9657	0.7585	0
混合召回Top20→bge-reranker-base重排	0.8627	0.9755	0.9804	0.8988	272.7

类型	重排前R@1	重排后R@1	重排前R@5	重排后R@5
事实	0.5	0.9722	1.0	1.0
事实-年份混淆	0.6667	1.0	1.0	1.0
同义改写	0.8889	1.0	1.0	1.0
对比	0.3182	0.2727	0.8636	0.8182
政策	0.6667	0.7778	0.8889	1.0
汇总	0.0	1.0	1.0	1.0
表格	0.8696	0.8696	0.9565	1.0

阈值	无答案题拒答率	有答案题误拒率
0.05	0.0	0.0
0.1	0.0833	0.0
0.15	0.0833	0.0
0.2	0.0833	0.0
0.3	0.1667	0.0
0.4	0.25	0.0
0.5	0.25	0.0

拒答策略（检索侧，不调用 LLM）	无答案题拒答率	有答案题误拒率
仅重排分阈值 0.15	8.3%	0.0%
阈值 + 查询约束校验（年份/季度/主体/指标）	83.3%	0.0%

4.6 端到端（0.6B 为 CPU 链路验证；最后一行 Qwen3-8B 走项目三 vLLM 服务）

检索侧（阈值 + 约束校验）不变，所以拒答率与误拒率与 0.6B 一致；换到 8B + vLLM 后，答案准确率 59.8% → 86.0%、引用准确率 53.7% → 89.5%、无出处数字 33.3% → 6.1%，生成延迟由 CPU 的 2938ms 降到 520ms，剩余错误集中在对比题与年份混淆题。

配置	答案准确率	引用准确率	拒答率	误拒率	无出处数字比例	生成ms
约束校验 + 模型可自行拒答	36.3%	77.3%	91.7%	52.9%	28.1%	2722.4
仅阈值 + 模型可自行拒答	36.3%	77.3%	91.7%	52.9%	28.1%	2946.2
约束校验 + 拒答决策前置（模型不给拒答出口）	59.8%	53.7%	83.3%	0.0%	33.3%	2938.2
Qwen3-8B（项目三服务）	86.0%	89.5%	83.3%	0.0%	6.1%	520.0

bad case 分桶（0.6B，约束校验）：{'误拒答(证据已召回)': 54, '生成幻觉(数字无出处)': 9, '生成错误(证据在但答错)': 2, '幻觉(无答案题未拒答)': 1}。最大问题是“误拒答（证据已召回）”——0.6B 模型过度保守，证据里明明有数字也回答“无法回答”。对策是拒答决策前置：检索侧（阈值+约束）判定可答时，生成 prompt 不再给模型拒答出口，只要求从证据中抽取。

4.7 故障记录（面试必讲）

故障	badcase 示例	根因	怎么修的	效果
空召回/错召回	“宁川 25 年 Q3 收入多少”	简称 + 口语时间与文档表述不一致	时间归一化 + 同义词 + 标题前缀	R@5 → 96.6%
表格被切断	“2026 年 EPS 预测”召回到半张表	固定长度切块切断表格	表格保护：表格独立成块、超长表重复表头	关闭保护 86.8% vs 开启 96.6%
双栏串行	左右栏同一行拼接成乱句	整页 extract_text	栏间空白带检测分栏	naive 86.8% → layout 96.6%
无答案题瞎答	“宁川动力董事长是谁”	证据主题相关、重排分高	查询约束校验（主体/年份/指标）	拒答 8.3% → 83.3%
约束误拒	“研报预测 2025 年 A 和 B 的净利润”被拒	把“研报预测”识别成公司名	通用前缀过滤	误拒率 → 0.0%
小模型过度拒答	证据里有数字仍答“无法回答”	0.6B 指令遵循弱、过度保守	拒答决策前置到检索侧	准确率 36.3%→59.8%
macOS 段错误	FAISS + torch 同进程崩溃	两份 libomp 冲突	KMP_DUPLICATE_LIB_OK + FAISS 单线程	稳定运行

项目三 推理部署 —— vLLM + 量化 + FastAPI 网关 + 压测与故障演练

一、项目介绍

1.1 项目一句话简介

> 把项目一的客服模型和项目二的研报 RAG 串成一个能扛并发的在线服务：vLLM 部署 AWQ-INT4 量化模型，FastAPI 网关负责意图路由、Redis 热点缓存、令牌桶限流、超时熔断与降级、日志监控；用并发阶梯压测、KV Cache 估算和故障演练给出 QPS/P95/显存与止血 SOP。

背景：为什么一定要有部署项目

- 只在 Notebook 里跑，面试官会觉得你只是“调参选手”。一问“服务 QPS 多少？P95 多少？OOM 了怎么处理？”十个人里八个答不上来。
- 场景接着客服来：日均 5000+ 请求，双 11 高峰 3 倍；要求单次响应 P95 < 2 秒、32 并发不崩；GPU 扛不住要有降级方案；热门问题（“双 11 能叠加优惠券吗”）直接走缓存不过模型。
- 做完这个项目，简历就不只是“会训练”“会 RAG”，而是“能跑通训练→检索→部署全链路”。

推理框架与技术选型

组件	选择	备选 / 理由
推理引擎	vLLM（PagedAttention、continuous batching、OpenAI 兼容 API、prefix caching）	SGLang（进阶 A/B）、LMDeploy（量化+部署一体，benchmark 对照）
量化	AWQ-INT4（AutoAWQ / llm-compressor），业务数据校准	GPTQ W4A16（llm-compressor）、INT8 W8A8（SmoothQuant）
服务框架	FastAPI 网关 + vLLM OpenAI-compatible server	—
缓存	Redis（多副本共享），不可用时自动退化为进程内 LRU	—
限流	令牌桶：单用户防刷 + 全局（只对打到 GPU 的请求计数）	Redis + Lua 分布式令牌桶（代码内附）
降级	熔断器（错误率 / P95 超 SLO）→ 小模型 + 短 max_tokens → 兜底话术 + 转人工	—
压测	自写异步压测（并发阶梯、TTFT、tokens/s、显存采样）	locust（附 locustfile）
监控	JSONL 请求日志 + /stats 滚动指标 + Prometheus /metrics（含 vLLM 自带指标）	Grafana
模型	项目一 Qwen3-8B SFT+DPO 合并后 AWQ-INT4	Qwen3-14B-AWQ（显存允许时）

整体路线：vLLM 跑通 → 量化对比 → FastAPI 套一层做工程化 → 压测 → 故障演练。产出一份量化对比表 + 一份压测报告 + 一份故障处理文档。

1.2 简历直接可用的项目描述（三种不同表述，已降重）

> **使用说明**
> 性能数字（QPS、P95、tokens/s、显存、掉点）全部来自 5090 单卡的实测：量化对比用 scripts/run_quant_bench.sh，并发压测用 bench/load_test.py（1/4/8/16/32 阶梯，TTFT、tokens/s、GPU 显存采样），准确率掉点用项目一同一份 400 条测试集测。KV Cache 容量是确定性计算，可直接引用；mock 环境的数字只用于验证网关逻辑，不要写进简历的性能指标里。

版本 A | 大模型推理服务部署与性能优化（vLLM + 量化）

项目定位：

将微调后的客服模型与研报 RAG 串联为在线服务，在单卡 32GB 预算下满足 32 并发、P95 < 2s 的 SLO，并具备高峰降级能力

核心方案：

- 基于 vLLM 部署 Qwen3-8B 客服模型，对比 FP16 / AWQ-INT4 / GPTQ-INT4 / INT8 四种精度的显存、吞吐、TTFT 与业务准确率，量化校准使用业务对话
- 搭建 FastAPI 网关：意图路由（客服→模型 / 研报→RAG）、Redis 热点缓存与预热、令牌桶限流、超时熔断与小模型降级、全链路 token/延迟日志
- 自研异步压测工具完成 1/4/8/16/32 并发阶梯压测，对比 max-num-seqs、prefix caching、chunked prefill 等调度参数
- 基于 GQA 修正的 KV Cache 公式做容量估算，并完成 OOM、延迟飙升、流量突增、热点集中四类故障演练

实验与结果：

- AWQ-INT4 相比 FP16：单请求 64→106 tokens/s（1.7×）、32 并发 1381→2133 tokens/s（1.5×），显存峰值 18.8GB→7.0GB，客服测试集准确率 81.5%→80.2%（掉点 1.3 个点）
- 32 并发下 QPS 34.41、P95 986ms；热点问题缓存命中后延迟 <1ms，缓存命中率 30%（叠加缓存后 32 并发 QPS 49.2）
- 估算 32GB 卡上 AWQ 权重约 4.6GB，可为 KV Cache 留约 24.2GB（约 17.6 万 token，≈86 条 2K 序列），FP16 仅能支撑约 44 条

技术栈：

vLLM、AutoAWQ/llm-compressor、FastAPI、Redis、httpx、Prometheus、Python asyncio

版本 B | LLM 在线服务网关与稳定性工程

项目定位：

为客服与研报问答提供统一推理入口，重点解决“高峰扛不住、慢请求拖垮整体、热点重复计算、故障无预案”四类线上问题

核心方案：

- 网关请求链路：单用户限流 → 缓存（归一化 query 为 key，个人订单上下文请求不缓存）→ 全局令牌桶 → 路由 → 带超时的模型调用
- 实现 CLOSED/OPEN/HALF_OPEN 熔断状态机：窗口错误率 >30% 或 P95 超 SLO 时切换降级小模型并缩短 max_tokens，半开探测自动恢复
- 研报路由接入 RAG 服务的约束校验结果，证据不足时网关直接拒答、不占用 GPU
- 提供 OpenAI 兼容透传接口（含流式）、/stats 滚动指标与 Prometheus 指标，并编写故障注入 mock 与演练脚本

实验与结果：

- 故障演练（mock 实测）：上游 30% 报错时熔断触发，请求全部由降级模型兜底，成功率保持 100%；故障清除后 4.9s 内自动恢复
- 3 倍突发流量下令牌桶拦截超额请求，放行请求 P95 1898.3ms；热点问题 100% 命中缓存且不消耗限流令牌
- 量化 + continuous batching 后 32 并发 P95 由 1522ms 降至 986ms

技术栈：

FastAPI、asyncio、Redis、令牌桶、熔断降级、Prometheus/Grafana、vLLM

版本 C | 客服 + 研报大模型服务化（训练→检索→部署全链路）
项目定位：
打通“微调模型 → 知识检索 → 在线服务”全链路，输出可落地的部署配置、容量评估方法与故障止血 SOP
核心方案：
- 合并 LoRA 并以业务数据校准量化，用项目一同一测试集评估量化掉点，保证训练侧与部署侧指标口径一致
- 拆解 prefill（计算密集）与 decode（访存密集）瓶颈，围绕 TTFT 与 tokens/s 双目标调整批处理与缓存参数
- 设计降级优先级：缓存热点 → 量化 → 调度参数 → 小模型兜底 → 话术兜底转人工
实验与结果：
- 单卡 32GB 下服务 32 并发，P95 986ms、吞吐 2133 tokens/s；AWQ 相对 FP16 显存节省 63%
- 形成量化对比表、压测报告、故障记录表与三项目串联架构文档
技术栈：
vLLM、AWQ/GPTQ、LMDeploy(对照)、FastAPI、Redis、locust、Prometheus

1.3 面试直接可背的项目介绍（1-2 分钟）

约 2 分钟
第三个项目是把前两个项目部署上线。场景还是电商客服：日均 5000 多请求，双 11 高峰 3 倍，要求 P95 两秒以内、32 并发不崩，GPU 扛不住要能降级，热门问题直接走缓存。
第一步是量化。我把项目一的 LoRA 合并后分别做了 AWQ-INT4、GPTQ-INT4 和 INT8，校准数据用的是业务客服对话而不是通用语料，然后在 vLLM 上对比显存、吞吐、首 token 延迟，并用项目一同一个测试集测准确率掉点，最后选了 AWQ。算一下就知道为什么：Qwen3-8B FP16 权重 16.4GB，32G 卡留给 KV Cache 的只有 12.4GB，只能撑 44 条 2K 长度的并发；AWQ 权重 4.6GB，KV Cache 有 24.2GB，能撑 86 条——同一张卡多服务一倍的人。而且 decode 是访存瓶颈，INT4 权重读得更快：32 并发 P95 从 1522ms 降到 986ms。
第二步是服务化，我用 FastAPI 写了网关：先做单用户限流，再查 Redis 缓存，缓存没命中才消耗全局令牌，然后按意图路由——客服问题走微调模型，研报问题先调项目二的 RAG 服务，如果约束校验不过就直接拒答不占 GPU。模型调用有超时和熔断，错误率超过 30% 或者 P95 超过 SLO 就切到小模型，并缩短 max_tokens，再不行就兜底话术转人工。
第三步是压测和故障演练，并发 1 到 32 阶梯压测记录 QPS、P50/P95/P99 和显存，演练了 OOM、慢请求、流量突增和热点集中。演练里我还发现一个设计问题：热点问题被全局限流误伤了，其实缓存命中几乎零成本，所以把全局限流挪到了缓存之后。

1.4 项目英文简历描述

English version
LLM Inference Serving & Optimization (vLLM + Quantization + Gateway)
• Served the fine-tuned Qwen3-8B customer-service model with vLLM; benchmarked FP16 vs. AWQ-INT4 vs. GPTQ-INT4 vs. INT8 on memory, throughput, TTFT and task accuracy, using in-domain calibration data.
• Built a FastAPI gateway with intent routing (chat model vs. RAG service), Redis hot-query caching with warm-up, token-bucket rate limiting, timeouts, a circuit breaker with small-model fallback, and token/latency logging with Prometheus metrics.
• Developed an async load tester (concurrency 1–32, P50/P95/P99, TTFT, tokens/s, GPU memory) and ran chaos drills (upstream errors, slow decoding, 3× traffic bursts, hot keys).
• Estimated KV-cache capacity with a GQA-aware formula: AWQ frees ~24.2 GB for KV cache on a 32 GB GPU (~86 concurrent 2K-token sequences vs. ~44 with FP16); achieved 34.41 QPS and 986 ms P95 at 32 concurrency (2133 output tokens/s, 7.0 GB peak).
Tech: vLLM, AutoAWQ, llm-compressor, FastAPI, Redis, asyncio, Prometheus

二、面试方法

2.1 引导提问的思路、方向与话术

部署方向的追问更偏工程和系统，画风和训练/RAG 不一样。介绍里要主动报“数 + 公式 + 取舍”，把面试官引到你最熟的 KV Cache 估算和故障演练上。

想引导的方向	钩子话术	准备好的另一半
KV Cache	“算一下就知道为什么必须量化”	公式 2×L×(kv_heads×head_dim)×seq×batch×bytes；Qwen3-8B 每 token 144KB；GQA 比 MHA 公式少 4 倍
量化取舍	“校准数据用业务对话而不是通用语料”	AWQ 保护激活大的显著通道；校准分布与线上一致掉点更小；用项目一测试集测掉点
瓶颈分析	“decode 阶段是访存瓶颈”	prefill 计算密集并行处理 prompt；decode 每步读全部权重和 KV，batch 越大越摊薄
稳定性	“热点问题被全局限流误伤了”	限流顺序调整：单用户限流→缓存→全局限流；演练前后对比
降级	“错误率或 P95 超 SLO 都会触发熔断”	状态机、窗口、最小请求数、半开探测、降级动作的优先级

2.2 追问应对、诚实边界与如何化解深挖

事实	可以怎么说	别说
mock 后端只验证了网关逻辑	“网关的熔断/降级/限流逻辑先在故障注入 mock 上做了单元级演练，性能数字是在真实 vLLM 上压的”	把 mock 的 QPS 当真实性能
单机单卡，没有 K8s 集群	“单机部署，多副本时缓存用 Redis 共享、限流用 Redis+Lua，扩容按压测得到的单卡容量线性估算”	“支撑了线上千万级流量”
没有真实双 11 流量	“用 3 倍突发流量做了压测模拟”	“经历过双 11”
没写 CUDA kernel	“用的是 vLLM/AWQ 的 Marlin kernel，理解原理但没自己写”	“优化了 CUDA kernel”

- 被问“延迟砍一半从哪入手”：按收益/成本排序——缓存热 query > 量化 > speculative decoding > batch 策略优化 > 换更小模型；并说清楚每一项预期收益怎么验证。
- 被问到没做过的（TensorRT-LLM、多卡张量并行）：讲原理 + 何时需要（单卡放不下 / 延迟要求更极致），并说明你的场景单卡 AWQ 已满足 SLO，所以没做。

2.3 面试常见 QA（45 题）

Q1. 一句话介绍项目。

vLLM 部署量化后的客服模型，FastAPI 网关串联 RAG，做缓存、限流、熔断降级与监控，并用压测和故障演练给出容量与 SOP。

Q2. 为什么选 vLLM？

PagedAttention 解决 KV Cache 碎片、continuous batching 提高 GPU 利用率、OpenAI 兼容 API、支持 AWQ/GPTQ/LoRA/prefix caching，社区事实标准。

Q3. PagedAttention 原理？

把 KV Cache 按固定大小的 block 分页管理，序列的逻辑块映射到不连续的物理块，类似操作系统虚拟内存；消除预分配最大长度造成的内部碎片，还能在并行采样/前缀共享时共享块。

Q4. continuous batching 是怎么回事？

不等一批请求凑齐、也不等整批结束：每个 decode step 调度器都可以把已完成的序列移出、把新请求插入，动态合并请求，GPU 利用率显著高于静态 batching。

Q5. Prefill 和 Decode 有什么区别？

Prefill 一次性并行处理整个 prompt，计算密集（矩阵乘），决定 TTFT；Decode 逐 token 自回归，每步都要读全部权重和 KV Cache，访存密集，决定 TPOT/吞吐。

Q6. P95 瓶颈通常在哪？

decode 阶段（访存瓶颈）和排队：并发高时请求在调度队列等待，长 prompt 的 prefill 会阻塞别人的 decode（可用 chunked prefill 缓解）。

Q7. KV Cache 显存怎么估？

2(K,V) × num_layers × hidden × seq_len × batch × bytes。GQA 模型要把 hidden 换成 num_kv_heads × head_dim：Qwen3-8B 是 36 层、8 个 KV head、head_dim 128，每 token 2×36×1024×2B=144KB，2K×32 条约 9GB。

Q8. 为什么 GQA 能省显存？

多个 query head 共享一组 KV head，KV Cache 与 KV head 数成正比；Qwen3-8B 32 个 Q head 只有 8 个 KV head，KV 缩小 4 倍，同显存可服务 4 倍并发。

Q9. 32GB 卡能服务多少并发？

FP16 权重 16.4GB，按 0.9 利用率扣掉激活预留，KV 只剩约 12.4GB≈9.0 万 token，约 44 条 2K 序列；AWQ 权重约 4.6GB，KV 约 24.2GB≈17.6 万 token，约 86 条（bench/kv_cache_calc.py）。

Q10. gpu-memory-utilization 是什么？

vLLM 预分配给权重 + KV block 池的显存比例；设太高和其他进程抢显存会 OOM，太低浪费并发能力。

Q11. max-num-seqs、max-model-len 怎么设？

max-model-len 决定单序列最长 KV；客服场景 4096 够用。max-num-seqs 是同时调度的序列上限，太小吞吐低、太大单请求变慢且易抢占。用压测找 P95 和吞吐的平衡点。

Q12. prefix caching 有什么用？

相同前缀（system prompt）的 KV 复用，客服场景每个请求共享长 system prompt，命中后 prefill 计算显著减少、TTFT 降低。

Q13. AWQ 原理？

Activation-aware Weight Quantization：根据激活幅度找出约 1% 的显著权重通道，用逐通道缩放保护它们，再做 group-wise INT4 量化，无需反向传播。

Q14. GPTQ 原理？和 AWQ 区别？

GPTQ 逐层用二阶信息（Hessian 近似）做最优量化并补偿误差（OBQ 思想）；AWQ 基于激活的缩放，校准更快、对校准集过拟合更少。实际效果接近，vLLM 上都有 Marlin kernel。

Q15. 量化掉了多少精度？

用项目一同一个 400 条测试集测准确率/自动解决率（run_quant_bench.sh 自动完成），报实测值；没有测过就说不出这个数，所以量化实验必须做。

Q16. 为什么 INT4 反而可能更快？

decode 是访存瓶颈，INT4 权重只有 FP16 的 1/4，读权重的带宽压力大幅下降；配合 Marlin 等高效 kernel，单请求 tokens/s 往往更高。但大 batch 计算密集时反量化开销会显现。

Q17. 校准数据为什么用业务数据？

量化误差取决于激活分布，用与线上一致的客服对话校准，显著通道选得更准；通用 wiki 文本校准可能保护了错的通道。

Q18. W8A8 和 W4A16 区别？

W4A16 只量化权重，计算时反量化为 16bit，省显存、提升 decode；W8A8 权重和激活都 8bit，可用 INT8 Tensor Core，计算密集的大 batch 场景更有优势，需要 SmoothQuant 处理激活离群值。

Q19. KV Cache 也能量化吗？

可以，vLLM 支持 FP8 KV Cache（--kv-cache-dtype fp8），KV 显存减半、并发翻倍，代价是轻微精度损失，需要用测试集验证。

Q20. 网关请求链路是怎样的？

单用户限流 → 缓存查询 → 全局令牌桶（只对打到 GPU 的请求） → 意图路由 → （研报：RAG 检索 + 约束校验） → 带超时的模型调用（熔断/降级）→ 日志与指标 → 写缓存。

Q21. 缓存 key 怎么设计？会不会答错？

归一化 query（去语气词、标点、空白）+ 路由做 MD5；只缓存通用 FAQ，带个人订单上下文的请求不缓存，避免“A 的订单答案给了 B”。热点问题启动时预热，TTL 一天。

Q22. 为什么限流要放在缓存之后？

演练发现热点问题被全局令牌桶误伤（命中率 100% 却有大量 429）。缓存命中几乎零成本，不应消耗 GPU 配额；所以单用户限流在前（防刷），全局限流在缓存之后。

Q23. 令牌桶和漏桶区别？

令牌桶允许一定突发（桶容量），平均速率受限；漏桶以恒定速率流出，平滑但不允许突发。在线服务通常用令牌桶。

Q24. 多副本时限流怎么做？

Redis + Lua 原子脚本实现分布式令牌桶（ratelimit.py 内附），或在网关层（Nginx/Envoy）做；缓存同样用 Redis 共享。

Q25. 熔断器怎么设计？

滑动窗口统计错误率和 P95；请求数 ≥ min_requests 时，错误率 >30% 或 P95 > SLO 打开熔断，所有请求走降级；open_seconds 后半开，放一个探测请求，成功则关闭、失败重新打开。

Q26. 降级策略有哪些？优先级？

① 切小模型（量化 4B）② 缩短 max_tokens ③ 话术兜底 + 转人工；更前置的是缓存与拒答（不占 GPU）。

Q27. 超时怎么设？

单次 LLM 调用 8 秒总预算，降级调用用一半预算；超时立即返回兜底话术，避免请求堆积拖垮整体。流式场景还要看首 token 超时。

Q28. OOM 了怎么办？

现象：进程被 kill / CUDA OOM。排查：nvidia-smi 看显存、检查 batch 与 max_model_len、是否有超长请求。修复：降 max-model-len、限制 max-num-seqs/max-num-batched-tokens、开 chunked prefill、降 gpu-memory-utilization 给其他进程留空间、换量化。网关侧熔断把流量切到降级模型。

Q29. 延迟突然飙升怎么排查？

看 vLLM 指标：num_requests_waiting（排队）、gpu_cache_usage_perc（KV 满了会触发抢占/重算）、TTFT vs TPOT 谁变大；常见原因是长请求占满 KV Cache 或流量突增。修复：限流、chunked prefill、长短请求分流。

Q30. 请求抖动 / 输出被截断？

抖动：GC、显存碎片、CPU 预处理瓶颈，可预热 + 固定 batch 参数；截断：max_tokens 太小或 finish_reason=length，调大 max_tokens 或流式输出。

Q31. 怎么压测？指标有哪些？

并发阶梯 1/4/8/16/32，每档固定请求数；记录 QPS、P50/P95/P99、TTFT、输出 tokens/s、错误率、429 数、显存峰值；prompt 用真实测试集保持长度分布。

Q32. 为什么 P95 比平均值重要？

SLO 关心尾部体验，平均值会被大量快请求掩盖；P95/P99 才反映排队和长请求的影响。

Q33. QPS 和并发是什么关系？

Little 定律：并发 = QPS × 平均延迟。并发上去后 QPS 先线性增长，接近 GPU 饱和后增长放缓、延迟上升，拐点就是容量上限。

Q34. 流式输出有什么好处？

首 token 很快返回，用户感知延迟取决于 TTFT 而非总时长；网关透传 SSE，同时统计 TTFT。

Q35. 如果要把延迟砍一半，你从哪入手？

缓存热 query（零成本）> 量化（访存瓶颈直接受益）> speculative decoding（小模型草稿+大模型验证）> batch 策略（max-num-seqs、chunked prefill）> 换更小模型。每步都用压测验证。

Q36. speculative decoding 原理？

小模型（或 n-gram/EAGLE 头）先生成 k 个草稿 token，大模型一次前向并行验证，接受的 token 直接输出；输出分布与大模型一致，在接受率高时显著加速 decode。

Q37. vLLM、SGLang、LMDeploy 怎么选？

vLLM 生态最全；SGLang 的 RadixAttention 前缀复用和结构化输出更强；LMDeploy TurboMind + AWQ 性能好、部署简单。可用同一压测脚本做 A/B。

Q38. 多 LoRA 怎么服务？

vLLM 支持 --enable-lora 同时加载多个 adapter，请求按 model 名路由到不同 LoRA，共享底座权重；适合客服/研报两个 adapter 共用一个底座。

Q39. 监控看哪些指标？

网关：QPS、P50/P95/P99、错误率、缓存命中率、429、熔断状态、input/output tokens；vLLM：running/waiting 请求数、KV cache 使用率、TTFT/TPOT 直方图；GPU：显存、利用率。

Q40. 日志记录什么？

每个请求：request_id、路由、是否命中缓存、served_by（主/降级/兜底/拒答）、input_tokens、output_tokens、latency、status、降级原因；JSONL 便于统计每小时 QPS、平均延迟和错误率。

Q41. 意图路由怎么做的？准确吗？

关键词规则优先（零延迟、可解释）：研报/财务/政策词 vs 客服词打分；生产上可换成 bge-small + 线性分类器。路由错误时 RAG 约束校验会拒答，客服模型也会说明无法处理，不会产生严重后果。

Q42. 研报路由为什么在网关就能拒答？

RAG 服务的 retrieve_only 接口返回约束校验结果（年份/主体/指标是否在证据里），不通过网关直接返回“资料中未找到”，不占用 GPU。

Q43. 怎么验证量化后效果没坏？

同一个测试集、同样的评测脚本、贪心解码：07_evaluate.py --backend openai 指向量化服务；对比准确率、复读率、转人工率，并抽查 badcase 是否新增。

Q44. 如何做容量规划？

压测得到单卡在 SLO 下的最大 QPS，按峰值（日常×3）÷ 单卡 QPS ×（1+冗余 30%）估算卡数；再用 KV Cache 公式校验并发上限。

Q45. 这个项目最大的收获？

性能问题要用数说话：先估算（KV 公式）再实测（压测）再演练（故障）；很多“优化”最便宜的是缓存和限流顺序这种系统设计，而不是改 kernel。

三、项目启动方法

```
cd project3_llm_serving
pip install -r requirements.txt                    # Linux + CUDA 12.x
# W9 合并 + 量化 + 对比（自动：合并→AWQ/GPTQ/INT8→逐个起 vLLM→测准确率→压测→汇总）
bash scripts/run_quant_bench.sh                    # 结果：results/quant_summary.md
# 单独起服务
bash scripts/start_vllm.sh awq models/qwen3-8b-cs-awq # :8000，served-model-name=cs-model
# W10 服务化 + 压测
redis-server --daemonize yes
uvicorn gateway.main:app --host 0.0.0.0 --port 9000
python bench/load_test.py --target openai  --url http://localhost:8000/v1 --stream --tag vllm_awq
python bench/load_test.py --target gateway --url http://localhost:9000 --hot_ratio 0.3 --tag gateway
bash scripts/run_vllm_param_bench.sh               # max-num-seqs / prefix caching / chunked prefill
python bench/kv_cache_calc.py --model qwen3-8b --seq_len 2048 --batch 32 --weights awq
```

```
# W11 故障演练
python bench/chaos_drill.py --gateway http://localhost:9000 --primary_admin http://localhost:8000
# W12 三项目串联
bash scripts/start_all.sh                    # vLLM + RAG(:8100) + Redis + 网关(:9000)
curl -X POST localhost:9000/chat -H 'Content-Type: application/json' -d '{"query":"双11能叠加优惠券吗"}'
curl -X POST localhost:9000/chat -H 'Content-Type: application/json' -d '{"query":"宁川动力2025年三季度毛利率是多少"}'
curl localhost:9000/stats

# 无 GPU 验证网关逻辑（mock 后端，支持故障注入 /admin/fault）
python mock/mock_llm_server.py --port 8000 --model_name cs-model --step_ms 12 --mean_out 60 &
python mock/mock_llm_server.py --port 8001 --model_name fallback-model --step_ms 8 --mean_out 40 &
REDIS_URL= uvicorn gateway.main:app --port 9000 &
python bench/chaos_drill.py
```

常见问题	解决
vLLM 启动 OOM	降 --gpu-memory-utilization / --max-model-len；确认没有其他进程占显存
AWQ 模型加载报 quantization 不匹配	AWQ 用 awq_marlin、GPTQ 用 gptq_marlin；llm-compressor 产物无需指定 --quantization
返回内容带 <think>	请求体加 chat_template_kwargs: {enable_thinking: false}（网关和压测脚本已默认添加）
Redis 未安装	网关自动退化为进程内 LRU（/stats 中 cache_backend 字段可见）
Windows 上跑 vLLM	vLLM 仅支持 Linux，Windows 请用 WSL2 + CUDA；网关/压测脚本可在 Windows 原生运行

四、项目实验数据

4.1 KV Cache 容量估算（确定性计算，可直接引用）

模型 / 权重精度	每 token KV	2K×32 并发 KV	权重显存	32GB×0.9 可用 KV	可服务 2K 序列数
Qwen3-8B FP16（GQA 8 KV head）	144 KB	9.0 GB	16.4 GB	≈12.4 GB（≈90K token）	≈44
Qwen3-8B AWQ-INT4	144 KB	9.0 GB	≈4.6 GB	≈24.2 GB（≈176K token）	≈86
若误用 MHA 公式（hidden=4096）	576 KB	36 GB	—	—	高估 4 倍

4.2 量化对比（5090 单卡实测）

准确率一律用项目一同一份 400 条测试集、同一评测脚本、贪心解码：FP16 的 81.5% 就是项目一 SFT+DPO(β=0.1) 的准确率，量化后的掉点才和项目一对得上。显存峰值为 32 并发下权重 + 实际 KV 占用 + 激活的实测口径。

方案	模型体积 GB	显存峰值 MB	单请求 tokens/s	32 并发 tokens/s	准确率(项目一测试集)	相比 FP16
FP16 原始	16.4	18800	64	1381	81.5%	基准：权重 16.4GB，32G 卡留给 KV Cache 的只有约 12.4GB（≈44 条 2K 序列）
AWQ INT4	4.6	7000	106	2133	80.2%	体积 -72%，单请求 1.66×，32 并发 1.54×，掉点 1.3 个点
GPTQ INT4	4.5	6900	101	2050	79.8%	体积 -73%，单请求 1.58×，32 并发 1.48×，掉点 1.7 个点
INT8 W8A8	8.7	11100	80	1805	81.0%	体积 -47%，单请求 1.25×，32 并发 1.31×，掉点 0.5 个点

4.3 并发压测（真实 vLLM，5090 单卡实测）

设置：AWQ-INT4 服务，流式输出，平均输出 62 token，每档固定请求数，记录 QPS / P50 / P95 / P99 / TTFT / 输出 tokens/s / GPU 显存峰值。AWQ 在 32 并发下 P95 986ms、QPS 34.41；FP16 同一档是 1522ms、QPS 22.28——FP16 虽然也在 2s SLO 内，但显存峰值 18.8GB、KV 余量只够 44 条 2K 序列，换 AWQ 后显存降到 7.0GB、P95 再降 35%，并发余量翻倍到 86

条：同样的卡多服务一倍的人，这才是量化的理由。

并发	QPS	P50 ms	P95 ms	P99 ms	TTFT P50	输出 tokens /s	显存峰值MB
1	1.71	572	619	671	45	106	6400
4	6.41	611	661	718	70	397	6600
8	11.78	665	720	781	105	730	6800
16	20.62	760	823	892	180	1278	6900
32	34.41	911	986	1070	310	2133	7000

网关叠加 30% 热点缓存后（results/loadtest_gateway_vllm_hot30.json）：32 并发 QPS 由 34.41 提升到 49.2，缓存命中率 30%，命中请求不经过 GPU，延迟 <1ms 且不消耗限流令牌；FP16 对照（results/loadtest_vllm_fp16.json）32 并发 QPS 22.28、P95 1522ms、显存峰值 18800MB。

4.4 网关逻辑验证（mock 后端，本机实测；仅验证逻辑，不代表 GPU 性能）

设置：mock 主模型单步 12ms、平均输出 60 token；30% 热点问题；研报问题经真实 RAG 检索服务（bge-small + reranker-base）。

并发	QPS	P50 ms	P95 ms	P99 ms	错误率	429
1	1.56	715.4	1279.0	1533.0	0.0%	0
4	7.68	531.7	1294.6	1442.1	0.0%	0
8	13.23	629.1	1405.4	1677.4	0.0%	0
16	15.88	967.2	1616.9	1694.7	0.0%	0
32	22.2	1104.8	1970.6	1988.0	0.0%	0

演练场景	请求数	成功率	P95 ms	服务来源分布	429	缓存命中	熔断原因
基线（无故障）	120	100.0%	1355.9	{'primary': 120}	0	0.0%	-
上游报错（模拟 OOM/5xx）	160	100.0%	624.9	{'fallback': 160}	0	0.0%	error_rate=0.30
解码变慢 6 倍（模拟 KV 抢占）	120	100.0%	5249.0	{'primary': 19, 'fallback': 101}	0	0.0%	p95=5841ms>2000ms
3 倍突发流量（96 并发）	600	20.5%	1898.3	{'http_429': 477, 'fallback': 123}	477	0.0%	-
热点问题集中访问	200	100.0%	54.2	{'http_200': 200}	0	100.0%	-

读法：上游报错时熔断器在错误率达到 30% 阈值后打开，请求全部由降级模型兜底（成功率 100.0%），故障清除后经半开探测 4.9 秒恢复；解码变慢时由 P95 超 SLO 触发熔断；突发流量下令牌桶拦截超额请求保护后端；热点问题在调整限流顺序后 100% 命中缓存、不再被误伤（调整前同一场景有 54% 的请求被 429）。

4.5 故障记录表

故障	现象	排查手段	修复方案
OOM	vLLM 进程被 kill / CUDA OOM，网关 5xx 激增	nvidia-smi 看显存；检查 batch、max_model_len、超长请求	降 max_model_len、限 max-num-seqs、开 chunked prefill、换量化；熔断切降级模型
延迟飙升	P95 从 1.x s 到 5 s+	vLLM num_requests_waiting、gpu_cache_usage_perc、TTFT vs TPOT	限流、continuous batching 参数、长短请求分流；P95 熔断降级
流量突增	排队、超时、错误率上升	/stats QPS 与 429 计数	令牌桶限流 + 缓存 + 降级；按压测容量扩容
热点被误伤	热点命中缓存却返回 429	演练日志：cache_hit 与 429 同时出现	全局限流移到缓存之后
请求抖动	部分请求偶发变慢	查 GC、CPU 预处理、显存碎片	预热 + 固定 batch 参数
输出截断	回答不完整	finish_reason=length	调大 max_tokens / 流式输出

故障	现象	排查手段	修复方案
RAG 服务不可用	研报类请求失败	网关日志 rag_unavailable	返回 503 明确提示，不让模型无证据作答

4.6 性能优化顺序（面试可直接说）

缓存（热点零成本）> 量化（decode 访存瓶颈直接受益）> Batching 与调度参数（max-num-seqs、prefix caching、chunked prefill）> speculative decoding > 硬件升级。每一步都要用同一压测脚本、同一 prompt 分布验证。

附录 通用经验：踩坑、简历自检、开源仓库

A.1 我踩过的坑，你们别再踩了

1. 只跑通不做实验——面试官问“你做了什么优化”，张嘴说不出东西，氛围直接冷下来。每个项目至少 3 组对比实验。
2. 用公开 demo 数据还不改——面试官一眼看出来，问“数据哪来的”就尴尬。本仓库的数据管线允许你接入公开数据、自己扩写、自己清洗，你要能报出“原始多少条、每条规则过滤多少、最后多少条”。
3. 三个项目互不相干——做了训练、RAG、部署但串不起来，整体感全无。本仓库项目三的网关直接调用项目一的模型和项目二的检索服务。
4. 只有代码没有文档——你能跑不代表面试官能看到你的思考过程，文档（实验表、badcase、故障记录）是你的证据。
5. 贪多嚼不烂——做 5 个浅的不如 3 个深的，面试不看数量看深度。
6. 不准备追问——项目做了但讲的时候结结巴巴，和没做区别不大。第二章的追问清单逐条过一遍。

A.2 做完之后怎么判断能不能写进简历（5 条自检）

自检项	标准	本仓库对应的证据
至少 3 组对比实验	不是“跑通了”，是“试过几种方案知道哪个好”	各项目第四章实验表
能说出口的数字	“效果不错”是废话，“准确率从 58% 到 82%”才是有效信息	results/*.json，均可复现
有翻车记录	面试官最想听你遇到问题怎么解决	badcase 报告、故障演练记录
能讲满 5 分钟不卡壳	背景→问题→方案→实验→结果→复盘	1.3 面试稿
追问三层接得住	为什么→换个方案会怎样→线上出问题怎么办	2.2 追问应对 + 2.3 QA

五条里不到三条的项目，说实话别往简历上放，写上去被问到反而减分。

A.3 讲 badcase 的万能结构

> 四步法
> 出了什么问题（现象+数据）→ 为什么（定位过程，看了哪些信号）→ 试了什么（至少两个方案，一个没用一个有用）→ 解决没（修复前后数字）→ 复盘（以后怎么提前避免）

A.4 开源仓库汇总

仓库	地址	用途 / 建议精读部分
LLaMA-Factory	github.com/hiyouga/LLaMA-Factory	SFT+DPO 全流程参考，数据格式规范、LoRA 配置、对齐算法实现（本仓库 configs/ 提供对照配置）
TRL	github.com/huggingface/trl	SFTTrainer / DPOTrainer，自写训练脚本的规范参考
PEFT	github.com/huggingface/peft	LoRA / QLoRA 实现，读 lora/layer.py
Transformers	github.com/huggingface/transformers	模型加载 / 推理 / chat template
Qwen3	github.com/QwenLM/Qwen3	基座模型文档 + 微调最佳实践（thinking 开关）
nanoGPT	github.com/karpathy/nanoGPT	理解训练闭环，建议做项目前花 2 天读源码
RAGFlow	github.com/infiniflow/ragflow	生产级 RAG，重点读 DeepDoc PDF 解析 + 检索融合 + 评测体系
Langchain-Chatchat	github.com/chatchat-space/Langchain-Chatchat	中文 RAG 经典参考：中文切块、向量库适配、混合检索

仓库	地址	用途 / 建议精读部分
QAnything	github.com/netease-youdao/QAnything	2-stage retrieval（Embedding 粗排 + Rerank 精排）与 rerank 阈值设计
FAISS / FlagEmbedding	github.com/facebookresearch/faiss ｜ github.com/FlagOpen/FlagEmbedding	向量检索 / BGE Embedding + Reranker
LlamaIndex / Milvus / GraphRAG	run-llama/llama_index ｜ milvus-io/milvus ｜ microsoft/graphrag	快速 baseline / 生产级向量库 / 多跳问答进阶
vLLM	github.com/vllm-project/vllm	推理事实标准，重点读 PagedAttention + continuous batching 调度器
LMDeploy / SGLang	InternLM/lmdeploy ｜ sgl-project/sglang	量化+部署一体化 benchmark 对照 / 进阶 A/B 对比
AutoAWQ / AutoGPTQ / llm-compressor	casper-hansen/AutoAWQ ｜ AutoGPTQ/AutoGPTQ ｜ vllm-project/llm-compressor	AWQ / GPTQ / W8A8 量化（AutoGPTQ 已停更，GPTQ 推荐 llm-compressor）

说法建议：“我参考了 RAGFlow 的文档解析思路 / LLaMA-Factory 的数据模板，但训练脚本、检索链路和评测是自己基于 TRL / FAISS 实现的”，比“我用 LLaMA-Factory 跑的”更容易被认可，也是事实（本仓库训练、检索、网关代码均为自写）。

A.5 三个项目合在一起时的简历排版建议

- 每个项目按“项目定位 → 核心方案（3-4 条）→ 实验与结果（2-3 个数字）→ 技术栈”四段写，和参考简历格式一致。
- 三个项目的数字不要互相矛盾：项目三量化后的准确率必须用项目一同一测试集测，“掉点 <1%”要和项目一的绝对值对得上。
- 项目名可以按目标岗位微调：投训练岗突出项目一、投 RAG/应用岗突出项目二、投推理/infra 岗突出项目三，顺序也随之调整。
- 同一份简历里三个项目用不同的“降重表述”（第一章 1.2 每个项目给了三版），避免句式雷同。